

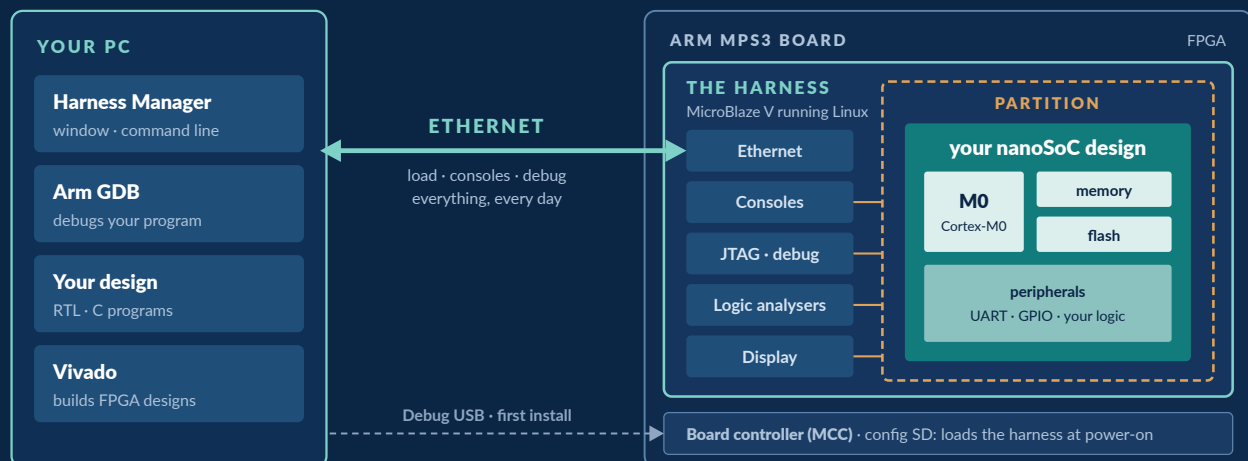
USER GUIDE

nanoSoC:

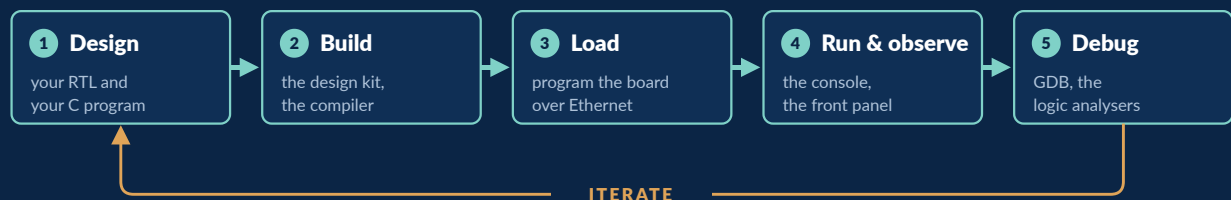
MPS3 Verification Platform

Everything after Getting Started, task by task: look after your board, load and keep designs, debug nanoSoC, run your own software, build your own FPGA designs, script it all, and share boards with others.

THE PLATFORM



HOW YOU WORK ON IT



Platform v2.0 · Harness Manager 1.0.0 · 7 October 2026

Companions: *Getting Started* and the *Technical Reference & Maintainers Guide*.

SoC Labs. Arm IP (Cortex-M processors, CoreSight, CMSDK) is used under licence from Arm and is not part of the platform.

Introduction

This guide is for you once you have worked through *Getting Started*: your board runs the harness, it is claimed with your key, and you have loaded a design and halted nanoSoC in a debugger. Here you go further, one task at a time.

With this guide you can:

- install, update, name and recover your own board, with or without a card reader;
- load, restore and keep designs, and understand what a swap does to the board;
- debug nanoSoC with its console, GDB and logic analysers, and put your own software on it;
- build your own design for the partition, check it and load it;
- script all of this, and share boards with other people.

Read chapter 1 once. After that, go to the chapter for your task: each one starts with what it achieves. The progress bar at the top of each page shows which stage you are in. The reference appendices at the back list the commands, ports, identities, designs and known limitations.

What you need first

- A board set up as in *Getting Started*, on your own network, and Harness Manager 1.0.0 on your PC.
- The Arm GNU Toolchain (`arm-none-eabi-gdb`, and `arm-none-eabi-gcc` for chapter 6).
- For the nanoSoC designs: Arm Academic Access and the nanoSoC source, which comes from SoC Labs, on proof of your Arm Academic Access (contact SoC Labs support). The download carries no nanoSoC design: you build each one you need with the kit (\$7.7). Everything else comes from <https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform/releases>.
- For chapter 7, and to build the nanoSoC designs: AMD Vivado 2026.1 with a licence for the board's FPGA.

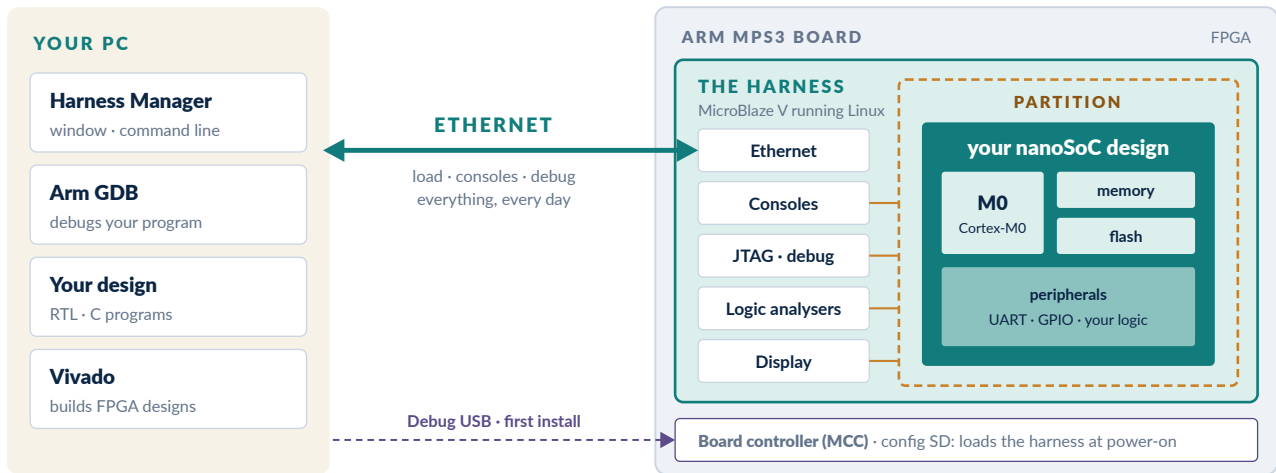
How to read the commands

YOU SEE	IT MEANS
<code>harness-manager info \$B</code>	A command to type. <code>\$B</code> is your board's address; set it once per terminal: <code>B=192.168.10.101</code> .
Grey text under a command	What it prints, from a real session. <code>...</code> marks lines left out.
Workbench, Board → Recover	A tab, page, card or button in the Harness Manager window.
<code>~/mps3-bundle</code> , <code>~/mps3-overlays</code>	The release you downloaded, and your designs, one directory each, as <i>Getting Started</i> set them up.
<code><design></code>	Something you fill in.

For help with a board that does not answer, or for the nanoSoC source, contact SoC Labs support (the support contact that came with your board).

The platform at a glance, and why

You know the pieces from *Getting Started*. This page is the map you will come back to; chapter 1 explains how each piece works and why it is built that way.



PIECE	WHAT IT IS	WHY IT EXISTS
The harness	A fixed FPGA design with a small Linux computer in it: the network, consoles, debugger, logic-analyser bridge and front panel.	It stays up while your design changes, so the board never needs a cable, a rebuild of the whole FPGA, or someone beside it.
The partition	The region of the FPGA that holds your design (the DUT, the device under test). Rewritten over Ethernet in about a minute.	One fixed boundary means any design built for it loads on any board running this harness.
nanoSoC	SoC Labs' family of Arm Cortex-M SoCs on an AHB bus fabric, the designs the harness is built around.	A real SoC you can read, change and take to silicon, with its console, debug port and flash wired through the harness.
The cards	The config SD holds the harness; the user microSD holds Linux, your claim, your board's name and a design to load at power-on.	Each card has one owner: the board controller reads the first, the harness the second.
Harness Manager	The window and the command line on your PC. <code>pyverify</code> is the Python library beneath it.	One tool does every task, by hand or from a script.

The flow on the cover is the loop you work in: design, build, load, run and debug, then change and go round again. Chapters 2 to 6 cover the board, the designs, debugging and software; chapter 7 builds your own designs; chapters 8 to 10 script the work, fix problems and share boards.

Contents

1	How the platform works	6	6.2	What a nanoSoC program looks like	28
1.1	The harness and the partition	6	6.3	Three ways onto the processor	28
1.2	Designs and their identities	6	6.4	MicroPython on nanosoc_upy	29
1.3	The two cards	7	7	Building your own FPGA design	31
1.4	How your tools reach the board	7	7.1	What you need	31
1.5	Harness Manager	7	7.2	What your design must provide	31
1.6	The front panel	8	7.3	The boundary rules	32
1.7	What to expect	9	7.4	Describe your design	32
2	Your board	10	7.5	Build, check and load	32
2.1	Your board's revision and MCC version	10	7.6	Build traps	34
2.2	Install or reinstall the harness	10	7.7	Worked case: building the nanoSoC designs	34
2.3	Put Linux on the user microSD with a card reader	11	8	The command line and scripting	36
2.4	Without a card reader: Linux over Ethernet	12	8.1	Harness Manager in scripts	36
2.5	Claim your board	13	8.2	pyverify	36
2.6	Name your board	14	8.3	Several boards from one script	37
2.7	OS slots and updates	14	8.4	OpenOCD on your PC: the fallback	37
2.8	Recover your board	15	9	Troubleshooting	39
2.9	Windows PCs	16	9.1	Reaching the board	39
3	Designs	17	9.2	Loading a design	40
3.1	The designs and where they come from	17	9.3	Consoles and debugging	40
3.2	Load a design	17	10	Sharing boards with others	42
3.3	What a swap does	18	10.1	Several boards on one network	42
3.4	Restore greybox	19	10.2	What a hub is	42
3.5	Keep a design on the card	19	10.3	Connect Harness Manager to the hub	42
3.6	The DUT clock	20	10.4	Leases and the house rules	43
4	nanoSoC in depth	21	10.5	Debugging on someone else's claim	43
4.1	What nanoSoC is	21	10.6	What else is different behind a hub	43
4.2	How nanoSoC meets the harness	21	A	Harness Manager commands	45
4.3	The memory map	22	A.1	Commands	45
4.4	How nanoSoC starts	22	A.2	Files and settings	46
4.5	GPIO, LEDs and switches	22	A.3	Exit codes	47
4.6	The variants in this release, and the flash cautions	22	A.4	Health states	47
5	Consoles and debugging	24	B	pyverify	48
5.1	The DUT console	24	C	Ports and protocol	48
5.2	Debug on the board with GDB	24	D	Identities and versions	49
5.3	By hand: the board's OpenOCD over SSH	26	E	Boundary signals	50
5.4	Logic analysers over XVC	27	F	The design catalogue	51
5.5	What the harness sees	27	G	Known limitations in v2.0.0	51
5.6	Limits in v2.0.0	27	H	The nanoSoC memory map	52
6	Software on nanoSoC	28			
6.1	The toolchain	28			

I **Glossary** **53**

1 How the platform works

This chapter explains what happens on the board when you use it, and why it is built that way. Read it once: every later chapter relies on these ideas, and knowing the reasons makes the rules easy to follow.

1.1 The harness and the partition

An FPGA is normally configured all at once, and everything on it stops while that happens. A board that reloads its whole FPGA for every new design loses its network, its consoles and its debugger each time, so you would need cables and someone beside it. The platform avoids that with AMD's partial reconfiguration, *Dynamic Function eXchange* (DFX), which splits the MPS3's FPGA into two regions:

- **The harness** is loaded from the config SD when the board powers up, and stays. It is a small computer built from FPGA logic: a MicroBlaze V processor running Linux, with the board's Ethernet, a JTAG bridge and a logic-analyser bridge into your design, the consoles and the front panel. Its service, `harnessed`, answers the network.
- **The partition** holds one design at a time, the **DUT** (device under test). The harness rewrites it over Ethernet while it keeps running.

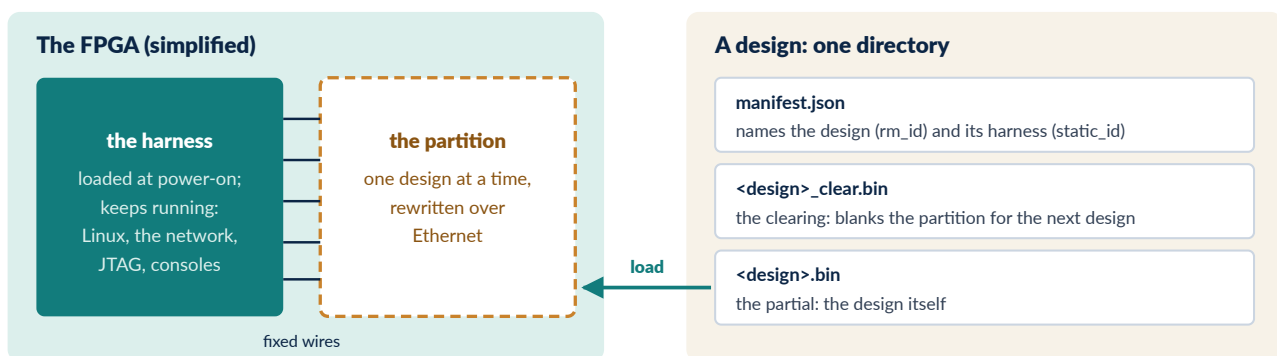


Figure 1.1. The harness, the partition and a design. Every design meets the partition on the same fixed wires, the **boundary**, so it loads only on the harness it was built for.

WHY

A design that hangs, or a bad load, can only ever break the partition. The harness keeps the network up, so you can always restore an empty partition, reset the DUT or reload, from your desk.

1.2 Designs and their identities

What you load into the partition is a **design**, also called an *overlay*: one directory with the three files of Figure 1.1. Designs that carry logic analysers also hold an `.ltx` probe file. **greybox** is the empty design: the state a board starts in and the safe state to return to. Harness Manager calls it *the baseline*.

Two numbers make loading safe:

- **static_id** fingerprints the harness: every v2.0 board reports `0x44EE76D5`. A design built for one harness does not fit another, so Harness Manager compares the manifest's `static_id` with the board's, and refuses a mismatch before it writes anything.
- **rm_id** is a constant inside each design. After a load the harness reads it back from the partition, and Harness Manager reports the load **verified** only when it matches. Verified means that what runs is what you sent.

Reference D lists every identity; reference F lists the designs of this release and their `rm_ids`.

1.3 The two cards

The board has two SD cards, each with one owner:

- **The config SD** belongs to the board controller, the **MCC** (motherboard configuration controller). It holds the harness, which the MCC loads into the FPGA at power-on. You write it over the Debug USB, where it appears as a drive named `V2M-MPS3`, only when you install or update the harness.
- **The user microSD** belongs to the harness. It holds Linux in two **OS slots**, A and B, so that a new image can be tried and rolled back; `/persist`, which keeps your claim, the board's name and its SSH host key; and the **design store**, which can hold one design to load at every power-on.

A board of your own needs the user microSD: without one, Linux must be sent from your PC after every reset, and the claim and the name are lost each time.

1.4 How your tools reach the board

Daily work needs only the Ethernet cable between your PC and the board (the figure on the "at a glance" page). The Debug USB is for installing the harness and for the MCC. The main services are:

SERVICE	PORT	WHAT USES IT
Control	6900	Every request: information, load, reset. One client at a time.
Design upload	6910	The files of a load.
DUT console	6930	Your design's UART, as a plain TCP byte stream.
SSH	22	The claim's key: the debugger, the logic analysers and changes to the user microSD go through it.
Identify	6899/udp	"What is at this address?", even when the board has no Linux running.

Reference C lists every port.

- **The debugger runs on the board.** OpenOCD, which drives the DUT's JTAG and serves GDB, runs in the harness's Linux next to the JTAG bridge. Your PC needs only GDB, and Harness Manager forwards its connection over SSH (chapter 5). *Why:* JTAG is a chatty protocol, and keeping it on the board avoids sending every bit across the network.
- **The claim.** The first SSH public key sent to a fresh board *claims* it: from then on the board trusts only that key, and the debugger, the logic-analyser port and the user microSD answer only through SSH with it. *Why:* those three give root-level control of the board and the DUT, so they must not be open to anyone on the network. Loading, restoring, resetting and the consoles stay open.
- **UDP stays on the local network.** Identify and the network boot of §2.4 use UDP, which works between your PC and the board but does not cross an SSH tunnel or a router. A firewall on your PC must let the replies in.

1.5 Harness Manager

Harness Manager is a window and a command line, `harness-manager`, that drive the same engine: anything you click can also be scripted. A background service on your PC holds the board connections; the window and the command line talk to it. The window has five tabs per board:

TAB	WHAT YOU DO THERE
Overview	The board's health, its loaded design and the Front panel card, a live mirror of the board's display.

TAB	WHAT YOU DO THERE
Workbench	Daily work: the Program the partition strip, the consoles, and a right-hand rail with the Debug and Logic analysers cards.
Build	Build your own design, in five steps: Setup, Design, Build, Check, Add (chapter 7).
Board	Six pages: Recover , Versions (the config SD and harness releases), Connections , Readings , Access (the claim and the board's name) and About .
Checks	Scripted board checks. It appears only for a board shared through a hub (chapter 10); for a board of your own, use a quick health check and a soak instead, as the <i>Technical Reference</i> describes.

`pyverify` is the Python library and command line underneath Harness Manager, for scripts and tests (chapter 8).

1.6 The front panel

The board's display shows what the harness is doing, so that you can check a board at a glance, or find it in a room. It has two pages: tap the panel's bottom row, or press the `USER_nPB1` button briefly, to turn the page.

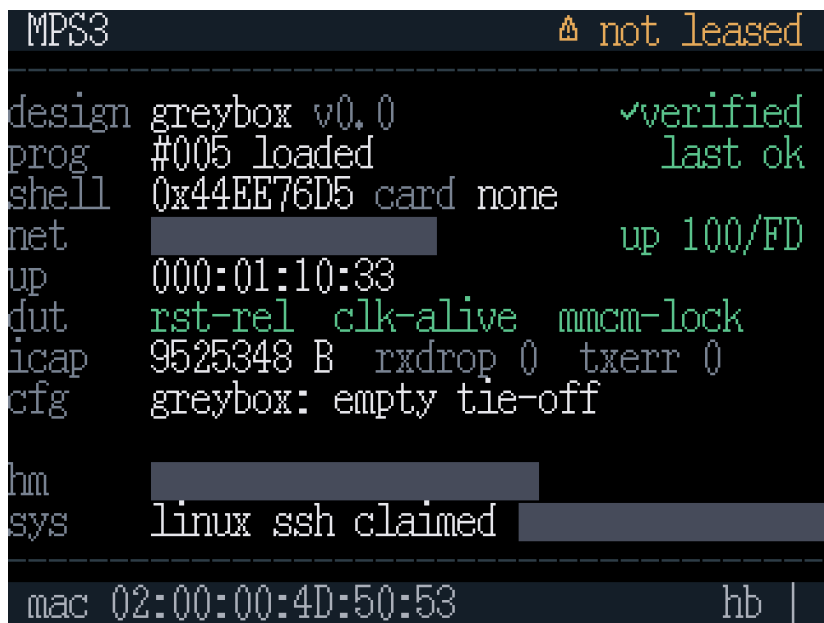


Figure 1.2. The status page, with greybox loaded. The board's address, the Harness Manager user and the key fingerprint are hidden here.

ROW	WHAT IT SHOWS
top	The board's own name (<code>MPS3</code> until you name it, §2.6). On the right, the hub lease, for a board shared through a hub.
design	The loaded design, its version and <code>✓verified</code> . Some designs show as <code>rm?<id></code> , such as <code>rm?01000005</code> for <code>nanosoc_upy</code> : a display limit of v2.0.0, not a fault.
prog	The last load (<code>#005 loaded</code> , <code>last ok</code>); during a load, its progress.
shell, net, up	The harness's <code>static_id</code> , and the design kept on the user microSD for power-on (<code>none</code> here); the address and the link; the uptime.
dut	The DUT's reset and clock: <code>rst-rel</code> (running), <code>clk-alive</code> , <code>mmcm-lock</code> .
icap, cfg	Bytes written by the last load and error counters; what the loaded design says about itself.
hm, sys	The Harness Manager user who has the board open (then <code>left 2m ago</code>); <code>linux ssh claimed</code> and the claim's key.
bottom	The board's MAC address and a heartbeat that moves while the harness runs.

- **During a load** the design row reads `partition decoupled`, the prog row shows the design and a percentage (`prog pushing led 55%`) over a blue progress bar, and the DUT row shows `rst-hold`.
- **Identify** (§2.6) turns three rows into a blue banner, `IDENTIFY: <user> via HM, tap here to say you found it`, and blinks the backlight.
- **The apps & ports page** lists how to reach the board: `ctrl nc <ip> 6900`, `tftp <ip> 69`, `push nc <ip> 6910`, `xvc <ip>:2542` and `ssh root@<ip>`, with the loaded design on its top row. Its footer reads `PB1 tap: next page hold: give DUT: holding the button hands the display to the DUT`, for designs that draw on it.
- Colours: green is good, amber is a warning, and white on red is a fault.
- **In the window**, the **Front panel** card on the **Overview** has two views. **Live** is the board's own picture, sent a few times a second (`live · software tap · 4 fps`); it is view only, so touch the board itself to turn the page. **Text** reads the rows as text you can select and copy (`Read from the panel, 45 s ago: ...`), which is handy for a bug report.

1.7 What to expect

- **A load takes about a minute** (35 to 86 s): `led` about 35 s, nanoSoC about 65 s, restoring greybox about 40 s. The files travel over the board's 10/100 Ethernet and are written into the FPGA as they arrive. It is not a hang.
- **The harness restarts slowly**. Linux takes 3 to 4 minutes to answer after a power-on or a reboot, and the partition stays in reset until the first load.
- **It proves function, not speed**. The DUT runs at 50 MHz, and one 10/100 link carries everything.
- **The boundary is fixed**. A design that needs its own clocks or a wide bus into the harness does not fit without a new harness (chapter 7).

2 Your board

This chapter covers looking after your own board: knowing which board you have, installing and updating the harness, putting Linux on the user microSD with or without a card reader, claiming and naming the board, the OS slots, and getting a board back when something goes wrong. *Getting Started* took you through the first install; here are the details and the other routes.

2.1 Your board's revision and MCC version

Check both before you install anything, so that you know what is supported.

- **The revision.** The silkscreen near the FPGA fan reads `HBI-0309B` or `HBI-0309C`. The file `LOG.TXT` on the config SD says the same, for example `MotherBoard Revision C Variant A`. "Variant A" is not a revision: that board is Revision C.
- **Support.** Revision C is supported. Revision B boots, but is untested: the harness bundle carries a folder for each revision, and the MCC reads the one for your board. There is no Revision A support.
- **The MCC firmware: check it, don't change it.** Open `LOG.TXT` and find the line with the MCC firmware version. This release is tested with v1.3.2; other versions are allowed, but untested. Harness Manager never writes the MCC's firmware, and refuses a bundle that would make the MCC update itself.

WHY

The MCC's firmware is outside the platform. Updating it is not needed for the harness, and a failed MCC update can leave a board that will not start without Arm's recovery tools.

2.2 Install or reinstall the harness

The harness lives on the config SD. You install it once per board, and again for each new release, over the Debug USB. In the window this is **Board** → **Versions: Back up the SD, Install onto the configuration SD, Reboot and witness it**, and **Restore the backup**. From a terminal:

- 1 **Connect and look.** Plug in the Debug USB. The config SD appears as a drive named `V2M-MPS3`, and four serial ports appear. `probe` lists the board and says which port is the MCC's.

```
harness-manager probe
```

- 2 **Back up the config SD.** It writes one zip and prints its sha256. Keep the zip off the board: it is your way back.

```
harness-manager sd - --volume /media/$USER/V2M-MPS3 backup ~/mps3-backups
```

3 **Install the bundle.** It asks first, refuses without a backup, and never writes the MCC's firmware. The write can take 5 minutes: slow is not failed, so do not unplug, power off or start a second write.

```
harness-manager sd - --volume /media/$USER/V2M-MPS3 install ~/mps3-bundle/config-sd \
--backup ~/mps3-backups/<the zip from step 2>

written    4 files to the SD of mps3@usb:/media/.../V2M-MPS3
not running yet: reboot (`harness-manager mcc TARGET reboot`), then check `harness-manager
info TARGET`
```

4 **Reboot from the MCC**, so that the board loads the new harness. It asks, then waits for the board to go down and come back.

```
harness-manager mcc - --serial /dev/ttyUSB0 reboot

rebooted    mps3@usb:serial:///dev/ttyUSB0 (seen: sent, down, up)
```

5 **Check it over Ethernet** after 3 to 4 minutes: `harness-manager info $B`. With Linux on the user microSD, expect `idle`. On a blank microSD, expect `rescue`: the harness is there and waits for Linux (§2.3, §2.4).

- as the board means "this board, over the Debug USB only". The names differ by system:

YOUR PC	THE MCC'S SERIAL PORT	THE CONFIG SD
Linux	/dev/ttyUSB0	/media/\$USER/V2M-MPS3
macOS	/dev/cu.usbserial-...	/Volumes/V2M-MPS3
Windows	a COM port, such as COM7	a drive letter, such as E:\

- **"no permission to open serial port"** on Linux means your user is not in the port's group: `sudo usermod -aG dialout $USER`, then log out and in again.
- **An interrupted install** leaves a marker, and the next install refuses until you restore the backup: `harness-manager sd - --volume /media/$USER/V2M-MPS3 restore ~/mps3-backups/<zip>`. The window shows an "Interrupted SD install" banner.
- **A firewall** on your PC can drop the board's UDP replies, which identify, the claim and the network boot need. Allow traffic from the board on that Ethernet port (on Windows, §2.9).

2.3 Put Linux on the user microSD with a card reader

The harness boots Linux from the user microSD. The quickest way to put it there is to write the whole card on your PC (about 15 minutes), as *Getting Started* does. The card image holds Linux in both OS slots and two empty partitions: `/persist` and the design store.

```
cd ~/mps3-bundle
python3 stage0_mkcard.py card --slot-a linux_slot.img --out-dir card \
--card-img card.img --card-mib 512
python3 stage0_mkcard.py check card.img      # ends: -> stage0 would boot slot A
```

Then write `card.img` to the whole microSD with `dd` (Linux, macOS) or a raw image writer (Windows), checking the disk name twice: it erases the card. Put the card in the board's user microSD slot and power on; Linux starts from slot A in 3 to 4 minutes. Claim the board next (§2.5): `/persist` is still unformatted, so this route needs the two claims too. **This route has not yet been run on a real board.** If it fails, use §2.4.

2.4 Without a card reader: Linux over Ethernet

This route does the same over the network, for a PC with no card reader. It was run end to end on a real board for this release. It takes about an hour and a quarter, most of it the push in step 4. Put the blank microSD in the board's user microSD slot first. You need `pyverify` and its **push tool**, `stage0_push.py`: with a checkout of the platform repository (<https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform>, chapter 8), `pyverify` finds it beside itself, in `src/linux_soc/hw/fw_stage0/`; otherwise point at it with `export MPS3_STAGE0_PUSH=<path>/stage0_push.py` or `--tool`. Without it, `netboot` stops with "STAGE0's push tool is not at ... (set `MPS3_STAGE0_PUSH` or `--tool`)".

- 1 **Start Linux from your PC.** A board with no Linux on its microSD waits in **rescue** for an image sent over the network. The image boots once, from memory.

```
pyverify identify --host $B # "mode": "rescue"
pyverify netboot ~/mps3-bundle/linux_slot.img --host $B # about a minute
```

```
netboot: accepted: stage0 verified the image and is booting it
```

About 20 s later `identify` says `"mode": "run"`. Exit codes: 0 accepted; 1 a bad image file; 2 refused by the board (`netboot --status` says why); 3 no board in rescue there; 4 the transfer failed.

- 2 **Claim it, for now:** `harness-manager board claim $B --key ~/.ssh/id_ed25519.pub`. Linux running from memory keeps this claim only until the reboot in step 5.
- 3 **Lay out the microSD, and format `/persist`. This erases the microSD.** The harness never lays out a blank card by itself. Make the layout on your PC, write its first three sectors through the board's SSH, then format:

```
python3 ~/mps3-bundle/stage0_mkcard.py card --slot-a ~/mps3-bundle/linux_slot.img --out-dir ~/mps3-card
harness-manager board ssh $B -c 'dd of=/dev/mmcblk0 bs=512 count=1 conv=fsync' < ~/mps3-card/mbr.bin
harness-manager board ssh $B -c 'dd of=/dev/mmcblk0 bs=512 seek=1 count=2 conv=fsync' < ~/mps3-card/bootsel.bin
harness-manager board ssh $B -c 'partprobe /dev/mmcblk0 && mps3-persist format --erase'
```

```
mps3-persist: formatted /dev/mmcblk0p3 (ext4, LABEL=mps3-persist); reboot to use it (mps3-reboot)
```

Run these one after another, with no restart in between. *Why*: on a reused card the old `/persist` sits where the new layout puts partition 3, and a restart before the format would bring back its old claim and host key.

- 4 **Push Linux into a slot** (about 45 minutes; leave it to finish). The board writes the image, then reads it back. It lands in **slot B**: the board runs from memory and slot A is the default, so the free slot is B.

```
harness-manager slot push $B --bundle ~/mps3-bundle
```

```
push: writing slot B: 0 MB / 32.6 MB, ~47 min left (70 KB/s)
...
push: verifying slot B (32.6 MB), ~39 min left (14 KB/s)
...
done      slot B holds linux_slot.img, read back; `harness-manager slot commit
192.168.10.101` makes it boot next
```

5

Commit and reboot. `slot commit` makes slot B boot next; then reboot from the MCC (with the Debug USB plugged in) or switch the board off and on. Linux is back in 3 to 4 minutes.

```
harness-manager slot commit $B
harness-manager mcc - --serial /dev/ttyUSB0 reboot
```

6

Check, and claim again for good. The first claim was in memory, so the board is unclaimed, with the host key it now keeps:

```
harness-manager slot status $B
harness-manager board claim $B --key ~/.ssh/id_ed25519.pub --replace-host-key
```

```
slots      mps3@192.168.10.101:6900: running B, default B, a push goes to A
B          valid ... booted, confirmed healthy, release ... [running, default]
```

Slot A stays empty, with no fallback, until your next push. Load a design, or restore greybox, once: after any restart the partition stays in reset until the first load.

2.5 Claim your board

A claim decides which SSH key the board trusts (§1.4). *Getting Started* claimed your board; this section explains what a claim is, how long it lasts and what to do when something changes.

CAUTION

The first public key sent to the board becomes the only key it trusts, for good. Nothing on the network can undo a claim kept on the user microSD; only `mps3-unclaim`, typed at the harness serial console, releases it. Claim with the key you will keep. If you pass a file with several keys, all of them are sent, although the question names only the first.

- **A new microSD takes two claims.** The board keeps the claim in `/persist`, which nothing formats by itself. Until it is formatted, the claim and the SSH host key live in memory and end at the next restart. So: claim, format `/persist` (`harness-manager board ssh $B -c 'mps3-persist format --erase'`), restart, claim again with `--replace-host-key`. `board ssh $B -c 'mps3-persist status'` says where it is: `backing=card` lasts; `backing=tmpfs` does not.
- **Claim only while info says idle.** In rescue the board takes an upload as a boot image.
- **In the window: Board → Access, Claim with my key...**, then **Claim with my key**.
- **Claimed another way** (with `pyverify claim`, or from another PC)? Pin it in this Harness Manager once: `harness-manager board claim $B --adopt`. It proves your key logs in and pins the host key. It adds no key.
- **A new host key** (a new card, a fresh `/persist`, or a restart while `/persist` is in memory) makes Harness Manager refuse the board. Compare `board claim-status $B` with `pyverify identify --host $B (host_key_sha256)`, then run `board claim $B --replace-host-key`.
- **To release a claim**, open the harness serial console: lane 2 of the Debug USB's four ports, 115 200 baud (on Linux, `screen /dev/ttyUSB2 115200`). Log in as root and run `mps3-unclaim`; the board reboots, unclaimed.

`harness-manager board ssh $B` opens a root shell on the harness with the pinned host key; `-c '<command>'` runs one command.

2.6 Name your board

Every board shows its own name at the top of its front panel, so that you can tell boards apart on a desk or in a lab. A new board is called `MPS3`, at `192.168.10.101`, with MAC `02:00:00:4D:50:53`. Give each board a name, and for two or more boards on one network, a unique address and MAC as well (chapter 10).

```
harness-manager board identity $B # what it is now, field by field, with each
one's source
harness-manager board identity $B --label MPS3-DESK --mac 02:00:00:4D:50:54 --ip 192.168.10.102/24
```

- **Names** use A–Z, 0–9 and `-`, at most 16 characters, so that they fit the panel. Use a locally administered MAC, one that starts `02:`, different on every board.
- It needs your claim and a user microSD (the name is kept in `/persist`). It asks you to type the new label, restarts the harness (about 3 minutes) and reads the result back. `--clear` returns to the defaults.
- **After a new address**, use it: `B=192.168.10.102`. Your PC must have an address on the same /24 network.
- **In the window: Board** → **Access**, the **Board identity** card, **Fix identity**....
- **Identify** blinks the panel backlight and shows `IDENTIFY: <you> via HM`, so someone in the room can confirm which board you mean: the magnifier icon on the board's card in the window, or `harness-manager identify $B --seconds 20` (1 to 30 s; 5 by default).

2.7 OS slots and updates

The user microSD holds Linux twice, in slots A and B. A new image goes into the slot not in use, boots only after you commit it and reboot, and a rollback brings the other back. *Why*: a bad image can never leave you without a Linux that boots. If a new image never boots healthy, the board goes back to the other slot by itself.

COMMAND	WHAT IT DOES
<code>slot status \$B</code>	Which slot runs, which is the default, where the next push goes
<code>slot push \$B --bundle DIR</code>	Write the bundle's Linux image into the free slot and read it back (about 45 min)
<code>slot commit \$B</code>	Make the pushed slot boot next. Between a commit and the reboot, no slot is free
<code>slot rollback \$B</code>	Make the other slot the default and reboot into it (<code>--no-reboot</code> : at the next reboot)
<code>slot verify \$B</code>	Read a slot back off the card: minutes, and it holds the card meanwhile

Updating to a new release. Releases are published at <https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform/releases>; Harness Manager's are at <https://github.com/SoC-Labs/HarnessManager/releases>.

- 1 **Read the release notes.** If the release changes the harness's `static_id`, every design must be rebuilt for it, including your own; the release brings new builds of the published designs.
- 2 **Update Harness Manager first**, with its installer: download the new release, or `git pull` in your clone, and run `scripts/install.sh --with-serial` again. Your settings are kept. (Harness Manager 1.0.0 does not update itself.)
- 3 **Install the new config SD** as in §2.2: back up, install, reboot.
- 4 **Push the new Linux image** into the free slot, commit and reboot (§2.4 steps 4 and 5), then check `slot status`. A rollback is one command away.

Harness Manager's built-in update commands (`update` , `harness` , and the release list in **Board → Versions**) are not live in 1.0.0: update by hand as above.

2.8 Recover your board

The board is yours, so every step here is safe for it. Start at the top and stop at the first step that brings the board back. In the window, **Board → Recover** lists steps 1 to 5 (*Gentle to heavy: start at the top and stop at the first step that brings the board back.*), with what each step keeps and loses, and a line saying which steps this board can do now. In Harness Manager 1.0.0 the page underestimates two steps: **Back to greybox** takes about 40 s, and **Restart the shell** takes 3 to 4 minutes (the harness restarts through its watchdog without reloading the FPGA). Its **Power-cycle** step works only with a networked power plug (Shelly, Tasmota or NETIO) set in `boards.toml` ; without one it says so, and you use the switch.

Recover
steps 1-3 of 5 here

Versions
1.0.0

Connections
Debug USB: none (Ethernet ...)

Readings
DUT 50 MHz

Access
SSH claimed

About
18 features · 8 not here

Recover
Gentle to heavy: start at the top and stop at the first step that brings the board back.

- This board can do steps 1-3 now; not step 4 (the row says why).
- Step 5, Power-cycle: cannot, needs a networked power plug in `boards.toml` (Shelly, Tasmota or NETIO).

STEP	HEAVIER	DUT	CONSOLES	DESIGN	LEASE	FPGA	
1	Reset DUT ~1 s	lost	kept	kept	kept	kept	Arm Reset DUT
2	Back to greybox ~10 s	lost	kept	lost	kept	kept	Arm To greybox
3	Restart the shell ~20 s	lost	drops	held	kept	kept	Arm Restart
4	Reboot via the MCC ~190 s	lost	drops	lost	kept	reloads	Arm Reboot

Cannot: needs the Debug USB cable, a networked power plug, or harness firmware with 'mccif' or 'mcc_local' (net-protocol v0.18) Fix: Connect J8 (Debug USB) to the hub or to this PC. [See Connections](#)

Hover a step or a cell for the detail. Your hub lease survives every step.

Figure 2.1. **Board → Recover**: gentle to heavy. Each step shows what it loses (the DUT's state, the consoles, the design, the FPGA), and needs **Arm** ticked first.

- 1 Reset the DUT:** `harness-manager reset $B`. It restarts your design and touches nothing else.
- 2 Back to greybox:** `harness-manager restore $B` (chapter 3). It clears a misbehaving design.
- 3 Restart the shell:** `pyverify reboot --host $B --wait`. Linux restarts in 3 to 4 minutes, without reloading the FPGA. The design stays loaded but held in reset until you load once.
- 4 Reboot via the MCC:** `harness-manager mcc - --serial /dev/ttyUSB0 reboot`, with the Debug USB plugged in. The FPGA reloads from the config SD, and the harness answers again after about 190 s.
- 5 Power-cycle:** the switch, off for ten seconds.
- 6 Put the config SD back** from your backup (`sd ... restore`), reboot, then install again (§2.2). If the `V2M-MPS3` drive does not appear at all, use a card reader: power off, take the config microSD out, unzip the backup onto it, put it back.

After steps 3 to 6, load a design or restore greybox once: the partition stays in reset until the first load.

- Rescue after a power-on** (`no valid slot` , `slots exhausted` , `card has no stage0 slots`): the user microSD has no Linux that boots. Write it again (§2.3) or push it again (§2.4), then claim again.

- **No answer after a power-on**, and **identify** silent or in rescue with `ddr calib fail`: press `USER_nPB0`, the harness's reset button. If it persists, reseal the DDR4 module.
- **A key you no longer have** holds the claim: release it at the harness serial console (§2.5).

2.9 Windows PCs

Harness Manager runs on Windows as on Linux and macOS, with the same window and commands. Windows and Linux PCs are supported; Linux is the most tested. Four things differ: the network, the Debug USB's names, the driver, and typing commands in PowerShell. Harness Manager 1.0.0 does not check your PC's network settings, so set them once as below.

- **The network.** The Ethernet adapter cabled to the board needs a fixed address on the board's /24 (192.168.10.1, no gateway), a **Private** network profile, and an inbound rule that allows UDP from 192.168.10.0/24. *Why*: Windows treats a new network as Public, and Windows Firewall then drops the board's UDP replies: identify, the claim (a TFTP transfer) and the network boot of §2.4. The board's TCP ports need no rule, because your PC opens every connection. *Getting Started* 6.4, step 3 has the Settings route. If a Windows Security alert about Python was cancelled, its block rule wins over any allow rule: disable it in **Inbound Rules**.

FROM THE COMMAND LINE

PowerShell as Administrator; `Get-NetAdapter` names the adapter, here `Ethernet 2`.

```
Set-NetIPInterface -InterfaceAlias "Ethernet 2" -Dhcp Disabled
New-NetIPAddress -InterfaceAlias "Ethernet 2" -IPAddress 192.168.10.1 -PrefixLength 24
Set-NetConnectionProfile -InterfaceAlias "Ethernet 2" -NetworkCategory Private
New-NetFirewallRule -DisplayName "Harness Manager board UDP" -Direction Inbound `
    -Protocol UDP -RemoteAddress 192.168.10.0/24 -Action Allow
Get-NetConnectionProfile -InterfaceAlias "Ethernet 2" # NetworkCategory : Private
```

- **The MCC's COM port.** The Debug USB adds four COM ports, listed in **Device Manager** → **Ports (COM & LPT)** as **USB Serial Port (COMn)**. `harness-manager probe` says which one is the MCC's: with FTDI's driver, each port's serial number ends in a channel letter, A to D, and A is the MCC. Give that port as `--serial COM7` (for example) to `mcc`. Channel C is lane 2, the harness serial console of §2.5: open it at 115 200 baud in a terminal program such as PuTTY, because Windows has no `screen`.
- **The FTDI driver.** The four ports need FTDI's VCP driver, which Windows Update usually installs when you first plug the Debug USB in. If the ports are missing, or listed under **Other devices**, install it from `ftdichip.com` (VCP Drivers), then unplug and replug the Debug USB. Without the channel letters, `probe` cannot tell which port is the MCC's, and offers none.
- **The V2M-MPS3 drive.** Windows gives the config SD a drive letter of its own; File Explorer shows it under **This PC**. `probe` finds it by its name, whatever the letter, and lists it; Harness Manager never takes the board's other drive, `MBED MPS3` (the on-board debug probe). Give the letter as `--volume E:\`. If the drive does not appear, try another USB port or cable.
- **Typing this guide's commands in PowerShell.** Set names as `$B = "192.168.10.101"` and `$PORT = "COM7"`. Type a command that the guide splits with `\` on one line. Write `$HOME\.ssh\id_ed25519.pub` where the guide writes `~/.ssh/id_ed25519.pub`. In `boards.toml`, write paths with forward slashes, or in single quotes. The installer adds `%LOCALAPPDATA%\harness-manager\bin` to your user PATH: open a new terminal after installing.

3 Designs

This chapter covers the designs you load into the partition: where they come from, how to load one and check it arrived, what a swap does to the board and your tools, how to go back to an empty partition, and how to make a design load by itself at power-on.

3.1 The designs and where they come from

This release has 13 designs, all for this harness (reference F lists each one). They come in two sets, and only the first is in the download:

SET	DESIGNS	WHERE FROM
Open	greybox , led , uart_echo , regdemo_a , regdemo_b , clcd_demo , dbg_demo , socscope . No Arm IP: anyone may use them.	Built, in the release download's overlays/ , at https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform/releases
Arm IP	nanosoc , nanosoc_upy , nanosoc_ila , nanosoc_multicore , eth_ss . They contain Arm IP (Cortex-M processors, CoreSight, CMSDK), so no one may publish them built.	Not in the download. With Arm Academic Access, you build them yourself with the kit (\$7.7), from the nanoSoC source, which comes from SoC Labs, on proof of your Arm Academic Access (contact SoC Labs support)

Keep every design in one directory, ~/mps3-overlays , one subdirectory each, and tell Harness Manager once: in the window, **Settings** → **Harness & kits** → **Extra overlay directories, searched first**, or `harness-manager config set mps3.overlay_dirs ~/mps3-overlays`. If `HARNESS_MANAGER_MPS3_OVERLAY_DIRS` is set in the Harness Manager service's environment, it wins, and the Settings page says so (*Your value is hidden by \$HARNESS_MANAGER_MPS3_OVERLAY_DIRS ...*). It must hold `greybox` , because restoring needs it. A design you build goes into Harness Manager's own store when you add it (`kit pack --import`), so it needs no directory. `harness-manager overlays $B` lists what loads on this board and why any other does not; it marks the Arm-IP designs, and `nanosoc_multicore` as "writes the DUT's flash" (§3.2).

3.2 Load a design

In the window, open the board's **Workbench**. In **Design to program**, pick a design; the picker tags some, such as *ILAs*, *writes DUT flash*, *loaded now* and *baseline*. The preflight runs at once and shows its result as chips: `6 checks OK · static_usercode matches · ... · no mismatch, 1 unchecked (not a pass, does not block)`. Then, in **Program the partition**, tick **Arm** and click **Program**. A green box reports the result: `Programmed led 0x0100001E · 35 s in all · pushed 1.3 MiB in 31 s · 43 KB/s · verified ...`. **Restore baseline** loads `greybox` (§3.4); **keep it on the card** also makes the design load at power-on (§3.5).

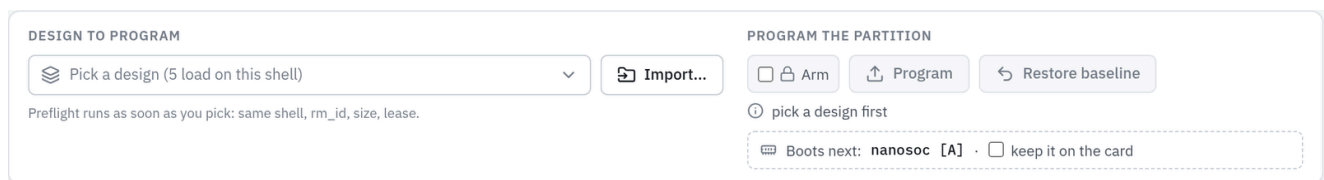


Figure 3.1. The top of the **Workbench**. **Arm** is a deliberate second step: a load replaces whatever runs in the partition.

From a terminal, `program` shows the same preflight and asks before it writes (`--yes` in a script):

```
harness-manager program $B nanosoc
```

```
ok          shell_id matches 0x44ee76d5
...
unchecked static_usercode matches needs JTAG ... (not a pass)
program nanosoc (0x01000001) into mps3@192.168.10.101:6900? [y/N] y
programmed nanosoc (0x01000001) into mps3@192.168.10.101:6900 in 65.0s via tcp; verified
```

The preflight checks that the board is free, that the design's `static_id` matches the board's, and that the files match their manifest. A design built for another harness is refused as `INCOMPATIBLE` (exit 14). `unchecked` lines are shown honestly and never count as a pass. A load takes about a minute; if it fails or is interrupted, restore greybox. Straight after a failed load the board can answer `busy` for up to 30 s.

THE BOARD'S OPENOCD STOPS FIRST

Before every load and every restore on a claimed board, Harness Manager stops OpenOCD on the board (`mps3-debug down`), whoever started it, so that nothing drives the DUT's JTAG while the partition is rewritten. Its own debug session reopens afterwards (§5.2). If OpenOCD cannot be stopped, nothing is programmed, and the command exits 15:

```
harness-manager: `mps3-debug down` failed before the swap (the board's SSH failed: ...): OpenOCD on
the board may still drive JTAG, so nothing was programmed – retry, or add --force to swap anyway
(on Linux v2.0.0 OpenOCD on the board may still drive JTAG during the reconfiguration)
```

The bracket says why: the board's SSH failed; OpenOCD's own exit (`exit 6, openocd_exit: ...`); it answered state `'up'`; or `no mps3-debug/1 answer`. Retry first. If it still fails, add `--force` to `program` or `restore`: the swap goes ahead with a warning instead (`deploy: warning: swapping anyway (forced) ...`). In the window, the refusal arms **Program anyway** (or **Restore anyway**): tick **Arm**, then click it. `--force` never takes a board that someone else holds.

NANOSOC_MULTICORE ASKS FOR A TYPED WORD

`nanosoc_multicore`'s boot code writes one byte into the DUT's flash at `0x20000` on every boot, which damages a MicroPython image there (§4.6). Harness Manager cannot see what the flash holds, so it warns before every load of it:

```
WARNING: nanosoc_multicore's boot code writes the DUT's QSPI flash (one byte at 0x20000 onward) on
every boot, until Linux v2.1: it damages a MicroPython image there (nanosoc_upy). Type MULTICORE
to program it anyway.
>
```

Type `MULTICORE` to go on. `--yes` never implies it: a script passes `--allow-dut-flash-write`, and a run with no terminal and no flag is refused (exit 15). In the window, the Program strip shows the warning and a field to type the word in.

3.3 What a swap does

A load is called a *swap*. Knowing its steps explains what you see, and what you must reconnect afterwards:

1. Harness Manager checks the design, stops the board's OpenOCD, and takes the board's control port for the whole load.
2. The harness holds the DUT in reset and *decouples* the partition: every output of the old design is clamped to a safe value. The panel shows `partition decoupled` and `rst-hold`.
3. It writes the clearing file, then the new partial, into the FPGA as they arrive over the network (the panel shows `prog pushing <design> 55%`).
4. It reconnects the partition, releases the reset, and reads the new `rm_id` back. A match is **verified**.

Everything that talked to the old design is now stale:

- **Consoles** are flushed. Reopen the console, then reset the DUT to see it start (§5.1).
- **The debugger:** Harness Manager's session reopens on new local ports, and the terminal holding `debug up` prints the new `attach` line (§5.2). Attach GDB again.
- **Logic analysers:** close the Vivado target before the load; wait 20 s after it, then reconnect and reload the probes (§5.4).

3.4 Restore greybox

Tick **Arm** and click **Restore baseline** (the green box reads `Restored baseline: greybox 0x00000000 · 39 s in all · ... · verified`), or:

```
harness-manager restore $B
```

```
restored mps3@192.168.10.101:6900 to the baseline (0x00000000) in 41.6s; verified
```

It takes about 40 s and asks no question, so it has no `--yes`. Like a load, it stops the board's OpenOCD first (`--force` as in §3.2), and it needs `greybox` in your design directories ("no baseline overlay (greybox) is known for the running shell" otherwise). Restore greybox when a design misbehaves, after a harness restart (the partition stays in reset until the first load), and when you finish, so the next session starts clean.

3.5 Keep a design on the card

With Linux on the user microSD, the harness can keep one design on the card and load it by itself at power-on, so the board comes up ready.

KNOWN ISSUE: THE POWER-ON LOAD DOES NOT COMPLETE IN V2.0.0

The harness reads the card too slowly at power-on: a `nanosoc` pair takes about 130 s against a 30 s limit, so `card status` shows `power-on failed:timeout` and the board starts on greybox. The kept design stays on the card; after a power cycle, load it again. A later release fixes this.

- 1 **Format the card's design store**, once. Until then `harness-manager card status $B` calls it `foreign`. A claimed board takes this only through its own SSH, so close the board in the window and forward its control port:

```
ssh -o ControlPath=none -N -L 127.0.0.1:16900:127.0.0.1:6900 root@$B &
pyverify usd format --host 127.0.0.1 --control-port 16900 # "state": "empty"
```

- 2 **Keep a design:** tick **keep it on the card** when you load it, or add `--keep-on-card`. After the load is verified, Harness Manager writes it to the card (about 4 more minutes for `nanosoc`) and reports `Kept on the card`, or why not. A failed card write never fails the load.

```
harness-manager program $B nanosoc --keep-on-card
```

- `card status $B` shows the kept design; `card commit $B` keeps the running one; `card clear $B` removes it, so greybox loads at power-on.
- Only a power-on loads the kept design. Hold `USER_nPB1` at power-on to start on greybox once.
- `pyverify deploy` keeps every verified load by default: pass `--no-persist` unless you mean it.

3.6 The DUT clock

Your design's clock, `dut_clk`, runs at 50 MHz. In v2.0.0 it is fixed: `harness-manager clock $B` reports `dut 50 MHz [pin-model]` (fixed by the shell: the Linux harness cannot change it), and **Board** → **Readings** shows "fixed by the shell". Build your design for 50 MHz; the nanoSoC console's 76 800 baud assumes it.

4 nanoSoC in depth

nanoSoC is the SoC family the harness is built around. This chapter explains what is inside it, how it sits in the partition, how it starts, which variants this release defines and how you get them, and the two flash cautions you must know before you load them.

4.1 What nanoSoC is

nanoSoC is SoC Labs' family of Arm Cortex-M microcontroller SoCs built on an AMBA AHB bus fabric, for research, prototyping and teaching, and designed to go to silicon. Members use different Cortex-M processors (Cortex-M0, Cortex-M0+ and Cortex-M4) around one structure: the processors, instruction and data memory, a boot ROM, peripherals on AHB and APB, CoreSight debug, and an expansion port on the bus for your own hardware. This release's designs use the Cortex-M0 (`nanosoc`, `nanosoc_upy`, `nanosoc_ila`) and the Cortex-M0+ (`nanosoc_multicore`). Its system is generated from a short description (a YAML file), so you can add memory, peripherals or an accelerator and regenerate it, rather than editing a fixed SoC by hand. On this platform the same nanoSoC that you would tape out runs in the partition, with its console, debug port and flash reaching you through the harness.

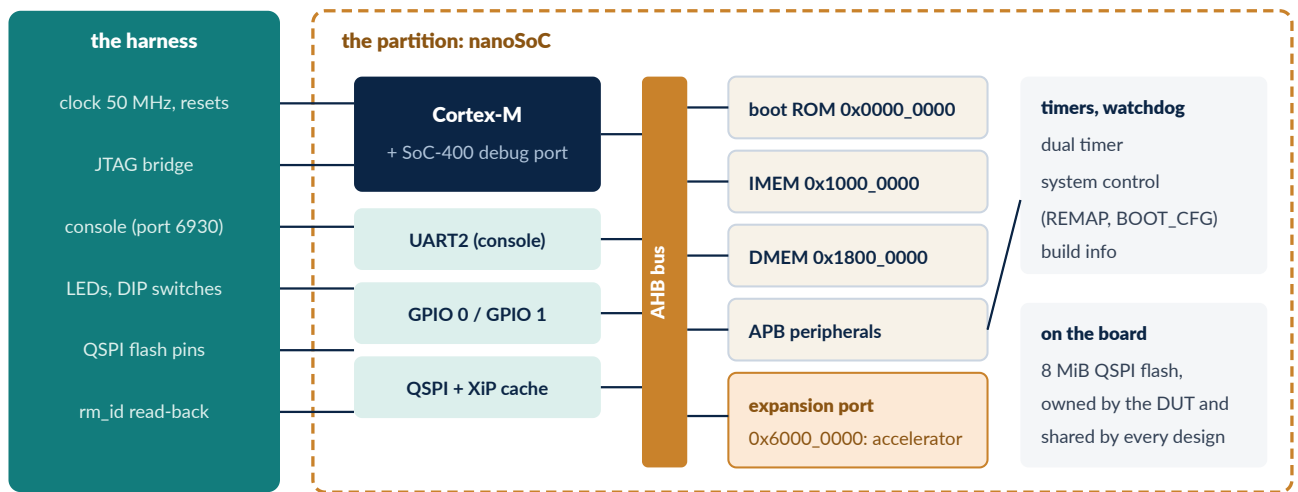


Figure 4.1. The single-core nanoSoC (`nanosoc`) in the partition, simplified. The harness gives it a clock, resets, a JTAG link, a console, the board's LEDs and switches, and the pins of the board's QSPI flash.

4.2 How nanoSoC meets the harness

A thin wrapper connects nanoSoC to the partition boundary (reference E). Knowing the connections tells you what to expect at your desk:

NANOSOC	THROUGH THE HARNESS
System clock	The 50 MHz <code>dut_clk</code> .
Reset	One reset, the AND of the harness's DUT reset, the load's reset and the debugger's reset. Every load and every <code>reset</code> starts nanoSoC from reset.
Debug	The CoreSight SoC-400 SWJ-DP, in JTAG mode, to the harness's JTAG bridge; the board's OpenOCD drives it (IDCODE <code>0x6ba00477</code>).
Console	UART2 (at <code>0x4000_6000</code>), turned into bytes for port 6930 at 76 800 baud . The firmware's clock constant says 25 MHz; at the real 50 MHz that gives 76 800, which is correct. Do not "fix" it.
GPIO 0	The board's eight LEDs on bits 7:0 and eight DIP switches on bits 15:8 (but see §4.5).
GPIO 1	Used inside nanoSoC by the console UART and its debug host interface, which the wrapper ties idle. Not available to you.

NANOSOC	THROUGH THE HARNESS
QSPI	The board's 8 MiB QSPI flash, wired straight through. The harness never writes it.
Identity	<code>rm_id 0x01000001</code> for <code>nanosoc</code> (reference F for the others).

4.3 The memory map

The single-core nanoSoC's main regions are below; reference H has the full table. For a SoC you generate yourself, the generated header `nanosoc_memmap.h` is the authority.

REGION	BASE	NOTES
Boot ROM	<code>0x0000_0000</code>	Runs at reset; IMEM replaces it at 0 after boot
IMEM, DMEM	<code>0x1000_0000</code> , <code>0x1800_0000</code>	Instruction and data memory, 16 KB each in <code>nanosoc</code>
APB peripherals	<code>0x4000_0000</code>	Timers, UARTs (console UART2 at <code>0x4000_6000</code>), watchdog
GPIO 0, system control	<code>0x4001_0000</code> , <code>0x4001_F000</code>	REMAP ; BOOT_CFG at +0x14
Expansion	<code>0x6000_0000</code>	256 MB AHB port: the slot for your accelerator
QSPI flash (XIP)	<code>0x7000_0000</code>	Execute-in-place window; controller at <code>0x7400_0000</code>

4.4 How nanoSoC starts

The Cortex-M0 has no register to move its vector table, so nanoSoC starts in a small **boot ROM** at address 0. The boot ROM prints a short marker on the console, then hands over to a program in instruction memory by setting the system controller's `REMAP` , which makes IMEM appear at address 0. Programs are linked to run from address 0 for that reason. Where that program comes from depends on the design:

- Built into IMEM** (`nanosoc`): the design's build fills IMEM with a program, here a hello program. It prints its banner once, at reset, then waits in a loop (`pc 0x378` under GDB). This is the dependable way to run software in v2.0.0 (chapter 6).
- From the DUT's flash** (`nanosoc_upy`): the boot ROM reads a boot table at flash address 0, copies the program's "hot" part (from flash `0x1000`) into IMEM, checks its CRC, sets up the execute-in-place window, and jumps. The "cold" part, from flash `0x20000` , runs in place at `0x7002_0000` . This is how a finished chip starts.

The `nanosoc` design's boot ROM also looks for a boot table in the flash, and uses it if one is there. That is why the flash matters to every nanoSoC design (§4.6).

4.5 GPIO, LEDs and switches

GPIO 0's sixteen bits reach the board: bits 7:0 drive the eight user LEDs and bits 15:8 read the eight DIP switches, through the harness. **In v2.0.0 the nanoSoC designs' GPIO 0 outputs stay at 0**, so the LEDs do not follow your program; the switches and the open `led` design are not affected. Use the console to see what your program does.

4.6 The variants in this release, and the flash cautions

DESIGN	WHAT IT IS	USE IT FOR
<code>nanosoc</code>	One Cortex-M0, 16 KB IMEM and DMEM, the hello program built in, an accelerator port	Your first debugging, and your own programs (chapter 6)

DESIGN	WHAT IT IS	USE IT FOR
<code>nanosoc_ila</code>	<code>nanosoc</code> with logic analysers on its console, GPIO, JTAG and reset, and an <code>.ltx</code> probe file	Seeing signals inside the SoC (§5.4)
<code>nanosoc_upy</code>	The nanoSoC with 128 KB of IMEM, booting MicroPython from the DUT's flash	MicroPython on the console (§6.4)
<code>nanosoc_multicore</code>	Two Cortex-M0+ cores, an Ethernet MAC with a PTP clock, DMA, inter-processor messaging and QSPI; the largest design	Multi-core and Ethernet work. Only core 1 can be debugged in v2.0.0

None of these designs is in the release download, because each contains Arm IP. With Arm Academic Access you build the ones you need, once each, from the nanoSoC source with the kit (§7.7).

The board's QSPI flash belongs to the DUT, not to a design: whatever one design writes there, the next one sees. Two known issues of v2.0.0 follow from that.

CAUTION: THE FLASH

Plain `nanosoc` locks up if the flash holds a MicroPython boot image. Its boot ROM loads the image, which is bigger than its 16 KB of IMEM, and the core locks up: no banner, and `pc 0xffffffff` in GDB; `monitor reset halt` does not clear it. A blank flash, or one without a boot image, works. If you have put MicroPython on the flash, use `nanosoc_upy`. A flash-erase step comes in a later release (known issue 14).

`nanosoc_multicore` writes the flash on every boot: one byte at `0x20000`, which damages a MicroPython image there. Do not load it on a board whose flash holds MicroPython; if you did, write the image again (§6.4). Harness Manager asks you to type MULTICORE before every load of it (§3.2; known issue 15).

`eth_ss`, the fifth Arm-IP design, is not a nanoSoC: it is an Ethernet MAC and PTP subsystem with no CPU, for testing the harness's virtual Ethernet link (reference F).

5 Consoles and debugging

The platform gives you four ways to look into a running design, all over the network. This chapter covers each: the DUT console, the debugger on the board with GDB on your PC, logic analysers in Vivado, and the harness's own view. It ends with the limits of this release.

USE	TOOL	WHEN
The console (§5.1)	Workbench console pane, <code>harness-manager console</code>	You want the firmware's own account of what it does
The debugger (§5.2)	GDB on your PC, OpenOCD on the board	The CPU is silent or stuck, or you need its registers and memory
Logic analysers (§5.4)	Vivado's hardware manager, over XVC	You need cycle-level truth about signals in the design
The harness (§5.5)	<code>info</code> , the front panel, <code>pyverify</code>	You suspect the harness, the link or a load

5.1 The DUT console

The DUT's console UART reaches you as a TCP byte stream on port 6930. In the window, the **Workbench's** console pane has a tab per stream: **DUT console**, `uart1` (inert on the published designs), **SWO** trace and **Harness; Reset DUT** (with **Arm**) sits beside them. A live DUT console says `up · uart0 · over Ethernet`. From a terminal:

```
harness-manager console $B uart0      # Ctrl-] exits; Ctrl-] then r, r resets the DUT
harness-manager pty $B uart0          # prints a screen command for the same console
```

- **Exit with Ctrl-],** not Ctrl-C: Ctrl-C goes to the board. **Ctrl-] then r** asks "reset the DUT of <board>? r or y resets it; any other key cancels"; **r** again resets it, and you stay in the console. That is the way to watch a program start from its first byte.
- **Open the console first, then reset the DUT.** The harness passes DUT output on only while a console client is connected, and every load flushes it: output that nobody reads is lost. The `nanosoc` hello banner is printed once, at reset, so a console opened afterwards shows nothing.
- **Output can drop bytes in bursts.** On the nanoSoC designs the harness holds only 16 bytes of console output at a time, so a burst longer than that can lose bytes. Print in short lines.
- **Pace your input.** The nanoSoC UART has no receive buffer: at most one character every 20 ms. Harness Manager paces what you type and paste for you; a raw reader does not.
- **The baud rate belongs to the design:** 76 800 for the nanoSoC designs (§4.2). It only matters for tools that check it; the stream itself is bytes.
- **The console holds the board's control port** while it is open, so `harness-manager reset $B` from another terminal is refused; use Ctrl-] r r instead. A raw reader such as `nc $B 6930` uses only the console port. `--read-only` watches without typing; `--export PORT` re-serves the console on `127.0.0.1` for a terminal program such as PuTTY.

5.2 Debug on the board with GDB

OpenOCD runs on the board, beside the DUT's JTAG, and Harness Manager forwards its GDB port to your PC over the claim's SSH. You need a design with a debug port (the nanoSoC designs; greybox and the open designs have none), `arm-none-eabi-gdb` on your PC, and your claim, made or adopted by this Harness Manager (§2.5).

In the window, the **Debug** card is in the Workbench's right rail. Open a session there: in about 50 s the card shows **up**, **OpenOCD on the board** and one GDB address per core (`gdb 127.0.0.1:43571`), with a copy button. Ports: `telnet on the board only · tcl on the board only` means OpenOCD's telnet and Tcl ports stay on the board's loopback (§5.3 forwards them by hand). While a session runs, the window's header can show **Last warning** for a harness read *over the 2 s read time*: the board's OpenOCD held the harness up for a moment, and it is harmless. **Detect** reads the debug port's IDCODE (`0x6ba00477`) with no reset and no halt; **Close session** stops OpenOCD. A session opened in the window needs no terminal: the Harness Manager service holds it.

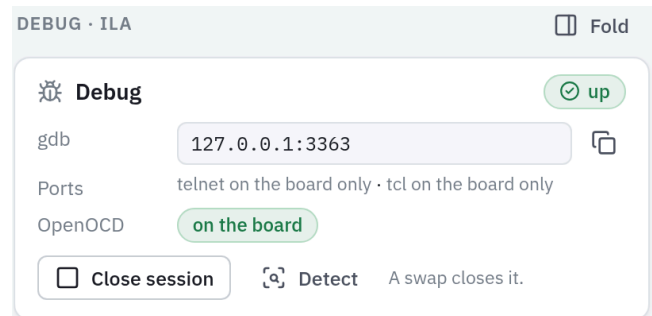


Figure 5.1. The **Debug** card with a session up.

From a terminal, `debug up` starts OpenOCD for the loaded design and holds the session in the foreground until Ctrl-C (`-- for 600` closes it after ten minutes):

```
harness-manager debug up $B

debug      mps3@192.168.10.101:6900: up
where      the board: OpenOCD runs there; gdb reaches it through the board's SSH
gdb        127.0.0.1:43571
attach     arm-none-eabi-gdb -ex "set remotetimeout 60" -ex "target extended-remote
127.0.0.1:43571"
config     nanosoc_mps3_jtag.cfg nanosoc_ops.tcl
...
```

In a second terminal (or after the window's copy button), paste the `attach` line on its own, once, then halt, look and detach. If GDB stops at `--Type <RET>` for more, `q` to quit, `c` to continue without paging`--`, type `c`.

```
arm-none-eabi-gdb -ex "set remotetimeout 60" -ex "target extended-remote 127.0.0.1:43571"
(gdb) monitor halt
(gdb) info registers pc sp
pc          0x378          0x378
sp          0x1800fbf0     0x1800fbf0
(gdb) detach
```

- **Copy the attach line, not just the port, and paste it alone.** The local port is a free one picked each time, and the line sets GDB's `remotetimeout` to 60 s: JTAG through SSH is slow, and with GDB's default of 2 s the attach fails. The line starts with `arm-none-eabi-gdb`: typing `gdb` before it starts your PC's own GDB instead, and pasting it twice glues the port to the next copy (`Servername not supported`). `keep_alive()` was not invoked in the 1000 ms `timelimit` warnings, 1 to 5 s late, are normal, after `monitor halt` and `monitor reset halt` alike. The first halt can take a minute.
- **Reading the registers.** `pc 0x378` is the hello program's final loop. `pc 0xffffffff` is a lockup: on nanosoc, the sign of a MicroPython image in the flash (§4.6). `monitor reset halt` alone does not clear that lockup: it prints `JTAG-DP STICKY ERROR` and `Deferring arp_examine`, and `pc` stays `0xffffffff`. Follow it with `monitor nanosoc.cpu0 arp_examine` and `monitor halt`: the processor stops in its boot ROM (`pc 0x0800...`), before it reaches the flash, and you can load a program (*Getting Started* ch 13). Arm GDB 13.3 has no `xpsr` register name ("Invalid register"); `info registers` lists them all.

- **One GDB port per core.** A two-core design gets two `gdb` and two `attach` lines, marked `cpu0` and `cpu1`. In v2.0.0 only core 1 of `nanosoc_multicore` can be debugged; the board refuses core 0.
- **Close the session** with Ctrl-C in the `debug up` terminal (about 20 s). `harness-manager debug down $B` stops a session the window opened. While a terminal holds `debug up`, other terminals' `debug down`, `program` and `restore` for that board are refused ("in use — your own `harness-manager debug up` (pid ...)` holds it"): press Ctrl-C there first.
- **For a script**, take the attach command from `harness-manager --json debug status $B` (`gdb_commands`, one per core) and run GDB with `-batch`.

A LOAD WHILE A SESSION IS OPEN

A load from the window while a terminal holds `debug up` is not an error. Harness Manager stops OpenOCD before the swap (§3.2), then reopens the session for the new design on new local ports, and the terminal follows:

```
swap      closed for the swap (OpenOCD on the board stops before a partition swap): the new ports
follow when it reopens
swap      reopened after the swap: the gdb ports below are new; attach gdb again
debug     mps3@192.168.10.101:6900: up
...
attach    arm-none-eabi-gdb -ex "set remotetimeout 60" -ex "target extended-remote 127.0.0.1:..."
```

Attach GDB again with the new line; `debug status $B` prints it too. If the board did not verify the load, the terminal prints `swap not reopened: the swap was not verified by the board` and holds no session: press Ctrl-C, then run `debug up $B` again.

WHEN DEBUG UP REFUSES

Each refusal names the cause and the fix: exit 13, the loaded design has no debug port; exit 4, another client holds the board's JTAG; exit 14, the board has no OpenOCD configuration for this design (only the published designs have one); exit 6, OpenOCD failed to start, with its last log lines; exit 15, this Harness Manager does not hold the claim (adopt it, §2.5). Chapter 9 has the details.

5.3 By hand: the board's OpenOCD over SSH

For OpenOCD's own console, or a tool Harness Manager does not drive, start OpenOCD on the board with its launcher, `mps3-debug`, and forward its ports yourself. These are the commands Harness Manager runs; by hand they have not yet been run on a real board.

```
ssh -o ControlPath=none root@$B mps3-debug up --rm nanosoc --json
ssh -o ControlPath=none -N -L 127.0.0.1:13333:127.0.0.1:3333 \
-L 127.0.0.1:14444:127.0.0.1:4444 -L 127.0.0.1:12542:127.0.0.1:2542 root@$B
ssh -o ControlPath=none root@$B mps3-debug down --json          # when you finish
```

Then attach GDB to `127.0.0.1:13333` (with `set remotetimeout 60`), or type OpenOCD commands into `telnet 127.0.0.1 14444`. `mps3-debug status --json` says when it is `"state": "up"` and which port each core has. OpenOCD's ports listen only on the board's own loopback, so nothing on the network reaches them; `-o ControlPath=none` keeps the forward off any shared SSH connection, where it would outlive your session. The board's OpenOCD loads helper procedures, which GDB runs with `monitor: nanosoc_halt_pc`, `nanosoc_cpuid`, `nanosoc_dump ADDR [N]`, `nanosoc_dump_imem [N]`, `nanosoc_reset_run` and `nanosoc_reset_halt`.

5.4 Logic analysers over XVC

A design built with logic analysers (ILAs) and a debug hub, such as `nanosoc_ila` or `dbg_demo`, ships an `.ltx` probe file. The harness's XVC server carries Vivado's JTAG traffic to them; it reaches only the partition's debug chain, never the whole FPGA. You need your claim and a `hw_server` from Vivado 2026.1 (the Lab edition is enough); an older one does not see these ILAs. (Not yet run end to end on a real board in this release.)

- 1 **Open the session.** In the Workbench's right rail, the **Logic analysers** card, under **Debug**: click **Open**. Harness Manager takes the board's one XVC slot and starts its own `hw_server` (tick **Bring your own hw_server** to use yours). From a terminal: `harness-manager xvc open $B`.
 - 2 **Connect Vivado.** Copy `Tcl` (or `harness-manager xvc tcl $B`) and paste it into Vivado's Tcl console: it connects and opens the target.
 - 3 **Load the probes.** The download button on the card's **Probes** line (or `xvc ltx $B -o nanosoc_ila.ltx`) gives the probe file for the loaded design. Then trigger and capture as usual.
 - 4 **Close** the card (`xvc close $B`) when you finish.
- **Close the Vivado target before any load**, and wait 20 s after a load before you reconnect: `hw_server` sees no devices meanwhile.
 - **The `.ltx` must come from the same build as the design.** The card's download always gives the matching one.
 - **One XVC client at a time.** Never probe the port with `nc`.
 - **ILAs see only what the design wires to them**, not arbitrary nets deep in the SoC.

5.5 What the harness sees

`harness-manager info $B`, the window's header and the front panel (§1.6) show the harness, the loaded design and the board's health: `idle` is ready; `busy` means another client holds the control port, or a load is finishing; `offline` means nothing answers; `rescue` means no Linux is running (§2.4); `wedged` means the harness took the connection and never replied (§2.8). `pyverify` reads more (reference B): `stats` the board in one line, `diag` the harness's counters, `log` its recent output, and `identify` the board's mode, claim and host key, even in rescue. The harness's software reports version `1.0.0` in this release; to tell releases apart, use the `static_id` (reference D).

5.6 Limits in v2.0.0

- **JTAG memory access is slow:** about 20 bytes per second through GDB, and a halt with a register read takes about 50 s. Read only the memory you need, and build programs into the design rather than loading them with GDB (chapter 6). **v2.0.1 or later is much faster**, with the same commands.
- **JTAG only.** The board's OpenOCD speaks JTAG; a design whose debug port speaks only SWD cannot be debugged.
- **Published designs only.** The board has OpenOCD configurations for the published designs and picks one by the loaded design's name; any other design is refused with exit 14. To debug a design of your own, contact SoC Labs support.
- **Writing the DUT's flash** through the board's OpenOCD is unproven (§6.3).

6 Software on nanoSoC

This chapter covers running your own programs on nanoSoC: the toolchain, what a program must look like to run there, the three ways to get it onto the processor, and MicroPython on `nanosoc_upy`. *Getting Started*, "Your own program", walks through one change, build and run; this chapter explains the choices behind it.

6.1 The toolchain

Use the Arm GNU Toolchain for `arm-none-eabi` (version 13.3.rel1 is the one tested with this release), from Arm's developer website. It gives you `arm-none-eabi-gcc` to build and `arm-none-eabi-gdb` to load and debug. Arm Development Studio also works: it connects to the same GDB port as a "Generic GDB" target.

6.2 What a nanoSoC program looks like

A program for this release's nanoSoC designs is a bare-metal Cortex-M0 (or, on `nanosoc_multicore`, Cortex-M0+) program. Four facts shape it:

- **Compile for the M0:** `-mcpu=cortex-m0 -mthumb -mfloat-abi=soft`. The M0 has no floating-point unit and no divide instruction; the compiler's library supplies both.
- **Link it at IMEM, `0x1000_0000`**, with the vector table first, as `examples/nanosoc_hello` does. GDB loads it there, and when the boot ROM hands over (§4.4) it also maps IMEM at 0, so the same image works built into a design. Data, the stack and the heap go in DMEM at `0x1800_0000`. In `nanosoc` each memory is 16 KB; keep the program within it.
- **The console is UART2 at `0x4000_6000`**, a CMSDK APB UART. At 50 MHz, a baud divider of 651 gives the 76 800 baud the harness expects.
- **Use the generated headers.** The nanoSoC generator writes `nanosoc_memmap.h` (every base address) and linker fragments for applications. Take addresses from them, not from memory: they are the authority for the SoC you built.

Print short lines, open the console before you start the program, and remember that the console can drop bytes in bursts longer than 16 (§5.1).

6.3 Three ways onto the processor

WAY	HOW	WHEN TO USE IT
Load with GDB	GDB writes the program into IMEM through the board's OpenOCD. The next load of a design removes it.	The edit, build, run loop. Quick in v2.0.1 or later.
Build it into the design	The program is an input to the design's build: it fills IMEM in the partial bitstream (chapter 7).	A program you want every load to start with; and in v2.0.0, any program larger than a few hundred bytes.
From the DUT's flash	The boot ROM copies it from the QSPI flash at every reset (§4.4).	Programs too big for IMEM, such as MicroPython, and boot as a finished chip does.

LOAD WITH GDB

With a debug session up (§5.2), stop the core in its boot ROM, load the program, point the core at it and run. This is the sequence *Getting Started* ch 13 uses, run on a board:

```
arm-none-eabi-gdb hello.elf -ex "set remotetimeout 60" -ex "target extended-remote
127.0.0.1:43571"
(gdb) monitor reset halt
(gdb) monitor nanosoc.cpu0 arp_examine
(gdb) monitor halt
(gdb) load
(gdb) monitor reg pc 0x10000040
(gdb) monitor reg sp 0x18004000
(gdb) monitor reg xPSR 0x01000000
(gdb) monitor resume
```

- **Speed.** In v2.0.0 GDB writes about 20 bytes per second, so a 1 KB program takes about 50 s; that is why this route is for v2.0.1 or later. The commands are the same in both.
- **Why these steps.** `monitor reset halt` can leave the core locked up if the DUT flash holds an image it cannot start (it prints `JTAG-DP STICKY ERROR`); `arp_examine` and `monitor halt` catch it in the boot ROM before it reaches the flash. The `monitor reg` values are your vector table's first two words (the stack pointer, and the reset handler without its Thumb bit) and the Thumb state bit; take them from your own ELF if it differs from the example.
- **A program that lands in `Default_Handler`** straight away met an interrupt left pending by the previous program; a Cortex-M0 cannot clear an active exception from the debugger. Stop SysTick and clear pending interrupts in your start-up code, or load the design again for a clean reset.
- A raw binary goes in with `restore hello.bin binary 0x10000000` instead of `load` (IMEM's own address).

The program runs (check it in GDB: reconnect and `print` a variable), but in v2.0 its console output does not yet reach the DUT console (appendix G).

BUILD IT INTO THE DESIGN

The nanoSoC designs read their IMEM contents from a word-per-line hex file when they are built. Point the design's build at your program's hex file (the `build.generics` entry in chapter 7's design file) and rebuild the design (about 50 minutes of Vivado). Every load then starts your program at reset, with no debugger needed.

FROM THE DUT'S FLASH

The harness never writes the DUT's flash. A **flash loader** does, running on the Cortex-M0 itself and driven over JTAG by an OpenOCD script (`firmware/qspi_loader/` in the platform repository, <https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform>). It needs a design with 128 KB of IMEM, `nanosoc_upy`, and an image made by `firmware/micropython/flash_pack.py`. In v2.0.0 it runs in an OpenOCD on your PC (chapter 8) and is slow: writing the MicroPython image took well over an hour. Writing the flash through the board's own OpenOCD is unproven. The full procedure, with every command, is in `fpga/rp/nanosoc_upy/README.md`. Mind the flash cautions of §4.6 before you write anything.

6.4 MicroPython on `nanosoc_upy`

`nanosoc_upy` boots MicroPython from the DUT's flash and gives you a Python prompt on the console. Build the design once first (§7.7; it is not in the download). After that, it is a quick way to poke at hardware with no further builds.

1

Check the flash holds the MicroPython image. Without it, `nanosoc_upy` prints a boot-ROM marker and stops. The image and the design's boot ROM are a matched pair: use the image from the same release.

2 Load the design, open the console, reset the DUT:

```
harness-manager program $B nanosoc_upy
harness-manager console $B uart0          # then Ctrl-J r r to reset and see the banner
```

3 Use the prompt. Type slowly or paste (Harness Manager paces input for you):

```
>>> print(1+1)
2
>>> import nanosoc
>>> nanosoc.switches()          # the eight DIP switches
>>> nanosoc.mem32(0x40010000)    # read a word; add a second argument to write one
```

`machine.mem8`, `mem16` and `mem32` give raw access to any address. `nanosoc.led()` writes the LED bits, but in v2.0.0 the nanoSoC designs' GPIO outputs stay at 0 (§4.5). The port has no filesystem and no floating point, to fit.

7 Building your own FPGA design

This chapter turns your own SoC, or a change to nanoSoC, into a design that loads on your board. Harness Manager writes the build script from the harness's **build kit**, you run Vivado once, and Harness Manager checks the result and adds it to your designs. The flow is AMD's standard non-project DFX flow, with no custom tools in it. Expect about ten commands and one Vivado run of 30 to 60 minutes.

7.1 What you need

YOU NEED	DETAILS
Vivado 2026.1	Exactly 2026.1, with the Kintex UltraScale device files (the FPGA is an <code>xcku115</code>). A design checkpoint opens only in the release that wrote it, and the build's first check refuses any other.
A Vivado licence for the KU115	The free tier covers neither this FPGA nor DFX. <code>vivado -version</code> succeeds without a licence, so it is no check: the build finds out at synthesis.
The Arm licence	Only if your design contains Arm IP, as every nanoSoC does. The kit contains no Arm IP; you read it from your own licensed copy.
The build kit	One per harness: the kit for <code>static_id 0x44EE76D5</code> is attached to the v2.0 release at https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform/releases . Download it from there.
A build machine	About 5 GB of memory at peak (allow 8 GB); 30 to 60 minutes per build, about 50 for a nanoSoC.

7.2 What your design must provide

Your design sits in the partition and reaches the harness only through the boundary (reference E), via a thin **wrapper**. Start the wrapper from the kit's skeleton, which declares every boundary port, drives a constant `rm_id`, and ties off the groups you do not use:

```
harness-manager xdc rm-kit --static-id 0x44EE76D5 --design my_soc.json --out x/ # writes
x/my_soc_wrapper_skeleton.sv
```

YOUR SOC PROVIDES	HOW THE WRAPPER CONNECTS IT
One system clock	<code>dut_clk</code> , 50 MHz.
One active-low reset	The AND of <code>dut_resetrn</code> , <code>rp_resetrn</code> and <code>dbg_resetrn</code> . Every load starts your design from reset.
A JTAG debug port	Generate the SoC with the CoreSight SoC-400 SWJ-DP (<code>EXTERNAL_DAP=1</code>), connect it to <code>jtag_tck/tms/tdi/tdo</code> , and strap <code>swj_enable=1</code> , <code>ntrst=1</code> . The board's OpenOCD speaks JTAG only.
A console UART	Through <code>uart_axis_shim.sv</code> (platform repository, <code>fpga/rp/nanosoc/</code>) to the boundary's byte streams, at 76 800 baud at 50 MHz.
GPIO <code>p0[15:0]</code>	To <code>dut_gpio_*</code> : LEDs on 7:0, switches on 15:8.
QSPI, for flash boot	Straight to the boundary's QSPI group (the one exception to the synchroniser rule below).
An identity	A 32-bit constant on <code>rm_id</code> : version major in bits 31:24, minor in 23:16, a design number in 15:0. <code>kit script</code> proposes one; give each design its own name and <code>rm_id</code> .

A standard nanoSoC's debug host interface on GPIO port 1 must be tied to its idle values, or the boot ROM's banner blocks the boot; copy the ties from the platform's `nanosoc` wrapper. If you work in a checkout of the platform repository (<https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform>), `fpga/rp/_template/` is the same skeleton for the in-repository build, with `new_rm.py` to copy and rename it.

7.3 The boundary rules

A wrapper that breaks one of these does not build worse: it fails, often after a full place and route.

RULE	WHY
No bus crosses the boundary: no AXI, no AHB	A wedged DUT must not hang a bus the harness needs. Only slow signals and byte streams cross.
No clock generation, no global clock buffers	A DFX restriction. The harness makes the DUT's clock.
No synchronisers of your own at the boundary	The harness synchronises every crossing on its side. QSPI is the exception: its read data is timed against the flash clock.
No pad registers (IOB, OLOGIC)	Pad sites belong to the harness in a DFX build.
Logic analysers through a debug bridge	A BSCAN primitive in the partition is illegal. Use a mode-1 debug bridge on the boundary's <code>dbgbscan</code> group, with its debug hub clocked from the 50 MHz <code>phy_rmii_ref_clk</code> , which never changes. Without ILAs, tie <code>dbg_bscan_tdo</code> to 0.
Every unused output has a constant	The build refuses a floating port. The skeleton already ties them off.
It fits	The partition has 42,824 LUTs and 144 block-RAM tiles; the single-core nanoSoC uses about 26 % of the LUTs (11,051).

The boundary is fixed for a harness release: changing it would make a new harness and force every design to be rebuilt.

7.4 Describe your design

Every `kit` command reads one design file, `my_soc.json`. List every source in compile order, packages first; read Arm IP in place, by absolute path, never copied. `build.generics` sets a top-level parameter, such as the hex file that fills IMEM (§6.3). To change a generated nanoSoC, change its YAML description and regenerate; never edit the generated files.

```
{
  "kind": "rm", "name": "my_soc", "rm_id": "<the id kit script proposes>",
  "use": {"clkfst": {}, "jtag": {}, "uart": {}, "status": {}, "gpio": {}, "qspi": {"timed": true}},
  "clocks": ["jtag_tck"],
  "wrapper": "rtl/my_soc_wrapper.sv",
  "build": {"top": "my_soc_wrapper",
    "sources": ["<packages first>", "...", "rtl/my_soc_wrapper.sv"],
    "include_dirs": ["build_soc/rtl"], "defines": ["RAM_PRELOAD"],
    "generics": {"IMEM_MEM_FPGA_IMG": {"path": "firmware/app.hex"}}}
}
```

Copy `use` and `clocks` from a built-in design (`harness-manager xdc info` lists them). The platform's own `nanosoc` design file, which comes with the nanoSoC source, is a complete example (§7.7).

7.5 Build, check and load

The window's **Build** tab runs the same flow in five steps: **Setup** (target, tools and kit), **Design**, **Build** (Vivado, 30 to 60 min), **Check** (the receipt) and **Add** (to the Workbench). From a terminal:

- 1 **Import the kit** you downloaded from the release, which checks every file and that its checkpoint matches the `static_id`; `kit info` shows the partition, the part and the Vivado release it needs.

```
harness-manager kit import <the kit zip from the release>
harness-manager kit info --static-id 0x44EE76D5
```

- 2 **Point your login shell at Vivado 2026.1** (its `settings64.sh` in `~/.bash_profile` or `~/.profile`, not only `~/.bashrc`), log in again, and check: `harness-manager config test tools vivado`.

- 3 **See where you are:** `harness-manager kit guide --static-id 0x44EE76D5 --design my_soc.json` lists the steps and names the next one. The licence shows as unchecked: only synthesis can check it.

- 4 **Write the build directory** (the script, a copy of the kit, the constraints, the skeleton). Copy the `rm_id` it proposes into your design file.

```
harness-manager kit script --static-id 0x44EE76D5 --design my_soc.json --out
~/builds/my_soc
```

- 5 **Get the Vivado command** (`kit build ~/builds/my_soc` prints it; it does not run Vivado; `--gui` gives the GUI form), **then run it**, exactly as printed. It writes the build's **receipt**. Watch for the end with a line-anchored match:

```
grep -E '^HM_RM_BUILD_(COMPLETE|FAILED)' ~/builds/my_soc/build_rm.log
```

- 6 **Check the result** (no Vivado, no board): the receipt, the files' CRCs, the bitstreams and the boundary timing (below). A failure exits 15.

```
harness-manager kit check ~/builds/my_soc
```

- 7 **Make it a design and add it:** `harness-manager kit pack ~/builds/my_soc --import` writes the manifest from the receipt; the design then appears in the Workbench and in `overlays $B`.

- 8 **Load it** as any other (chapter 3): `harness-manager program $B my_soc`. The preflight is all *ok* except `static_usercode`, which only JTAG can read; the load reads your `rm_id` back.

NOTE: THE VERDICT IS THE RECEIPT

Vivado exits 0 even when one of the build's checks fails. Each check prints an `HM_GATE` line, and `kit guide --why <gate>` explains one. The log echoes the script, so the text `HM_RM_BUILD_FAILED` can appear in a passing log; match whole lines only, and trust `kit check`. The flow reproduces SoC Labs' own builds: `nanosoc`, built with these commands on a pre-release of Harness Manager 1.0.0, was byte-identical to SoC Labs' build (§7.7).

THE BOUNDARY-TIMING CHECK

Vivado's summary can say every timing constraint is met while the paths between the harness and your design were never analysed (known issue 11). `kit check`, `kit pack` and the Build tab's **Check** step read the build's timing report and say which case you have. When the boundary was timed:

```
the static<->RM boundary was timed: ... found 0 register/latch pins with no clock and 0
unconstrained endpoints (423 more on a constant clock: not timing paths)
```

When it was not, a warning (never a refusal):

```
WARNING: boundary not timed (known issue 11): Vivado's check_timing in <rpt> found N
register/latch pins with no clock ... and M unconstrained endpoints (Harness Manager warns above 0
and 1,000), so the static->RM partition boundary was not timing-analysed, even where the summary
says every constraint is met. Your RM's own paths were still timed.
```

A build script written by Harness Manager 1.0.0 times the boundary, so you should see the first message. The warning means the build used a `build_rm.tcl` from an older Harness Manager, or a synthesis checkpoint you brought that already carries a `create_clock`. Its fix line says what to do: write the script again with `harness-manager kit script ... --out DIR` and rebuild. Such a design still loads, and its own paths were timed; but until you rebuild, keep margin on the signals that cross the boundary. The published v2.0.0 designs are not affected: they were re-timed and meet boundary timing (known issue 11, closed).

7.6 Build traps

SYMPTOM	CAUSE	FIX
The tools check finds another Vivado, or none, over SSH or in a new terminal	A login shell reads <code>~/.bash_profile</code> or <code>~/.profile</code> , never <code>~/.bashrc</code>	Put the <code>settings64.sh</code> line in the login-shell file; log in again.
About 20 CRITICAL WARNINGS in the Vivado log	Expected: tied-off ports and the harness's own debug bridge	None. The receipt is the verdict.
The build stops at the <code>sources_given</code> check, naming undriven outputs	The design file names no RTL; built-in names such as <code>nanosoc</code> carry none	Write your own design file with <code>build.sources</code> (§7.4).
<code>kit pack --import</code> says another design is listed instead	Another design has the same name, <code>rm_id</code> and <code>static_id</code> ; the first found wins	Give yours its own name or <code>rm_id</code> version.
<code>debug up</code> exits 14 on your design	The board has OpenOCD configurations only for the published designs	For a nanoSoC-based design, try an OpenOCD on your PC with the platform's nanoSoC configuration (§8.4); otherwise contact SoC Labs support.

7.7 Worked case: building the nanoSoC designs

The release download carries no nanoSoC design, because each contains Arm IP. With Arm Academic Access you build them yourself, once each, with this chapter's flow. *Getting Started* chapter 11 builds `nanosoc` step by step; this section explains the design file and covers the other designs.

What you combine. Four inputs, none of which the kit contains:

- the nanoSoC source and its design file, `nanosoc.json`, which come from SoC Labs, on proof of your Arm Academic Access (contact SoC Labs support);
- the Arm IP from your Arm Academic Access (Cortex-M0, CoreSight SoC-400, CMSDK and a few peripherals), read in place through a link named `arm_ip_library`;
- the platform files, in the nanoSoC source's `platform` folder: the wrapper `fpga/rp/nanosoc/rp_nanosoc_wrapper.sv`, its console shim, and `hello_image.hex`, the program built into IMEM;
- the build kit for `0x44EE76D5`.

IN NANOSOC.JSON	VALUE, AND WHY
<code>name, rm_id</code>	<code>nanosoc</code> , <code>0x01000001</code> . Keep both: the board picks its debugger set-up by the ID (§5.2), and the panel names the design by it. <code>kit script</code> warns that the ID is in the platform's own range; for this design that is expected.

IN NANOSOC.JSON	VALUE, AND WHY
<code>use, clocks</code>	Clock and reset, JTAG, UART, status, GPIO and QSPI (timed); <code>jtag_tck</code> . The same as the built-in <code>nanosoc</code> .
<code>wrapper, build.top</code>	The platform's <code>rp_nanosoc_wrapper.sv</code> , module <code>rp_nanosoc_wrapper</code> .
<code>build.sources</code>	246 files in compile order, SystemVerilog packages first: the nanoSoC source's FPGA file list, then five platform files (the console shim, the wrapper, and the display controller on the expansion port).
<code>include_dirs, defines</code>	Nine include directories (the generated SoC's and the Arm IP's); <code>RAM_PRELOAD</code> , so that IMEM starts filled.
<code>build.generics</code>	<code>IMEM_MEM_FPGA_IMG: {"path": "platform/fpga/rp/nanosoc/hello_image.hex"}</code> . Point it at your own program's word-format hex file to build that in instead (§6.3).

Every path is relative to the file's folder, so build from the command line (the Build tab's Paste .json needs absolute paths). First generate the boot ROM with `make -C nanosoc_m0_soc/pynq firmware` (Arm GNU toolchain and CMake), then build as in §7.5, with Vivado 2026.1 exactly:

```
cd ~/nanosoc
harness-manager kit script --static-id 0x44EE76D5 --design nanosoc.json --out ~/builds/nanosoc --jobs 4
harness-manager kit build ~/builds/nanosoc          # run the command it prints
harness-manager kit check ~/builds/nanosoc
harness-manager kit pack ~/builds/nanosoc --import
```

What to expect. With Harness Manager 1.0.0, SoC Labs' build took about 50 minutes with four threads and about 5 GB of memory. Every check passed, and the design's own paths met 50 MHz with 3.3 ns to spare. The boundary check reports `the static<->RM boundary was timed`: that is the pass you want. If your store already holds a `nanosoc`, the new one has the same name and ID: load it with `--overlay-dir`, or the old one hides it.

CAUTION: NEVER BUILD THE BUILT-IN NANOSOC

The built-in design named `nanosoc` (the Build tab's example, or `--design nanosoc`) describes only the partition's ports and timing: it contains no nanoSoC. Harness Manager 1.0.0 stops it at preflight (`sources_given`) and names the 13 outputs it would leave undriven. Releases before 1.0.0 built it as an **empty design** that passed every check and loaded under `nanosoc`'s name and ID. Build only from the design file, with Harness Manager 1.0.0 or later.

The other Arm-IP designs build the same way, each from its own design file and the wrapper in its platform directory (`fpga/rp/<design>/`). SoC Labs has not yet built them through the kit, so treat them as unproven; ask SoC Labs support for their design files.

DESIGN	WHAT TO KNOW BEFORE YOU BUILD IT
<code>nanosoc_upy</code>	Its boot ROM and the MicroPython flash image are a matched pair (§6.4).
<code>nanosoc_ila</code>	Its logic analysers need the <code>.ltx</code> probe file from the same build (§5.4).
<code>nanosoc_multicore</code>	The largest design: allow more time. It writes the DUT's flash on every boot (§4.6).
<code>eth_ss</code>	No CPU; CMSDK bus blocks only. For testing the harness's virtual Ethernet link.

8 The command line and scripting

Everything in this guide can run unattended. This chapter shows how to drive Harness Manager from scripts, how to use `pyverify` from a terminal and from Python for tests, and how to fall back to an OpenOCD on your PC. Reference A and B list every command.

8.1 Harness Manager in scripts

- **Output.** `--json` (before the verb) prints one JSON object; `--tsv` prints tab-separated rows whose columns are only ever appended, so a script keeps working across versions. Each command's `--help` lists its TSV columns.
- **No questions.** Commands that ask take `--yes`. Typed safety phrases are never implied by it: a load of `nanosoc_multicore` needs `--allow-dut-flash-write`, and a renamed board `--consent`.
- **Exit codes** say what went wrong (reference A.3): 4 held by someone else, 7 unreachable, 14 incompatible, 15 refused by a safety rule. A failure with `--json` is `{"ok":false,"error":{"code","name","message","hint"}}`.
- **The console in a script.** `console $B uart0 --read-only --for 10` prints ten seconds of output and stops.

A minimal check that a design loads and talks:

```
set -e
harness-manager program $B uart_echo --yes
harness-manager console $B uart0 --read-only --for 5 > console.txt &
sleep 1; harness-manager reset $B; wait
grep -q "rm_uart_echo ready" console.txt && echo PASS
harness-manager restore $B
```

If `reset` is refused because the console holds the board, read the console with `pyverify dut-console` instead (§8.2), which uses only the console port.

The service API. The window talks to the Harness Manager service over a local HTTP API on `127.0.0.1`, and so can your programs: the port and a bearer token are in `~/.config/harness-manager/daemon.json`, the base path is `/api/v1`, and long operations return a job id and report progress as events. `harness_manager.client.RemoteEngine` is a Python client. The Harness Manager repository's `docs/API.md` has the full list.

8.2 pyverify

`pyverify` is the platform's Python library and command line, and Harness Manager is built on it. Use it for tests: it drives the same board through the same ports, and a test written with it can also run with no board. Harness Manager's installer already put a copy in its own environment:

```
pyverify() { ~/.local/share/harness-manager/venv/bin/python -m pyverify.cli "$@"; }
```

For your own tests, the console bridge and the push tool of §2.4, install it from a checkout of the platform repository instead (Python 3.10 or newer):

```
git clone https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform mps3 && cd mps3
python3 -m venv .venv && . .venv/bin/activate && pip install -e host/pyverify
```

Then define `pyverify() { python3 -m pyverify.cli "$@"; }`. A session from a terminal:

```
pyverify identify --host $B          # "mode": "run", "impl": "linux", the claim and host
key
pyverify deploy --host $B --overlay ~/mps3-overlays/uart_echo --no-persist
pyverify dut-console --host $B --send "hello" --seconds 3    # types at 20 ms per character
pyverify reset --host $B
pyverify stats --host $B          # the board in one line: "rm" is the loaded rm_id
pyverify deploy --host $B --overlay ~/mps3-overlays/greybox --no-persist
```

Pass `--no-persist` to `deploy` unless you want the design kept on the card: unlike Harness Manager, `pyverify` keeps every verified load by default. `deploy` exits 0 loaded and verified, 1 refused, 2 a bad manifest, 3 a network failure.

In Python, `Mps3Board` is the session. This test loads `uart_echo`, takes its verdict from the DUT's own console, and leaves the board on greybox (not yet run on a real board in this release):

```
from pathlib import Path
from pyverify import Mps3Board

with Mps3Board("192.168.10.101", overlay_root=Path.home() / "mps3-overlays") as board:
    result = board.deploy("uart_echo", persist=False)    # check, load, verify
    assert result.verified
    board.uart0                                         # open the console before the reset
    board.reset("dut")
    board.uart0.assert_contains(b"ready", timeout=10)   # the verdict comes from the DUT
    board.deploy("greybox", persist=False)
```

- **The verdict comes from the DUT**, its console or its debug port, never from the harness's telemetry, which cannot see inside your design.
- **A load makes consoles stale:** `result.reattach.apply()` reopens them.
- **The same test runs with no board:** point it at `pyverify.testing.fakeshell.FakeShell`, a byte-exact model of the harness's network side.

8.3 Several boards from one script

Every command takes the board's address, so one script can drive several boards: give each board its own name and address first (§2.6), then loop over the addresses. Each board serves one control client at a time, so run the boards in parallel, not two jobs on one board.

8.4 OpenOCD on your PC: the fallback

OpenOCD normally runs on the board (§5.2). An OpenOCD on your PC, driving the board's JTAG port (6921), is the fallback: for a board you have not claimed, and when you set `debug.on_board` to `false`. With the default, `auto`, Harness Manager uses it by itself when the board's own OpenOCD cannot be used, and `debug status` then says where this PC: OpenOCD runs here, on the board's JTAG port (6921).

- 1 **Install an OpenOCD with the `remote_bitbang` adapter**, such as xPack OpenOCD 0.12.0-7. Check:


```
openocd -c "adapter list" -c shutdown 2>&1 | grep remote_bitbang
```

 prints a line.
- 2 **Tell Harness Manager:** `harness-manager config set tools.openocd <path>/openocd`, then `harness-manager config test tools`. After changing it, run `harness-manager daemon stop`; the next command starts the service again with the new tool.

By hand, with the configuration files in the platform repository's `host/openocd/`. On a claimed board, port 6921 answers only through SSH, so forward it first:

```
ssh -o ControlPath=none -N -L 127.0.0.1:16921:127.0.0.1:6921 root@$B &
openocd -c "set TRANSPORT_MODE rbb" -c "set RBB_HOST 127.0.0.1" -c "set RBB_PORT 16921" \
-f host/openocd/nanosoc_mps3_jtag.cfg -f host/openocd/nanosoc_ops.tcl \
-c "init; nanosoc_halt_examine"
```

- Put `-c "set ..."` before `-f`, or the configuration keeps its defaults. Start with `nanosoc_halt_examine`: a bare halt answers Target not examined yet.
- **One JTAG client at a time.** While the board's own OpenOCD runs, yours is refused. Leave 2 s between sessions.
- **jtag locked: board claimed (use ssh)** means OpenOCD connected to the board's address directly instead of through the forward.

The same configuration fits designs built on the nanoSoC's debug port; for anything else you need a configuration of your own.

9 Troubleshooting

Most failures on this platform look like hardware faults and are not. Find the symptom below: each row gives the cause and the fix. If nothing fits, go down the recovery ladder (§2.8). Exit codes are in reference A.3, and the known limitations of this release in reference G.

9.1 Reaching the board

SYMPTOM	CAUSE	FIX
<code>info</code> says <code>offline</code>	Your PC is not on the board's network; the cable or the power; or the board is still booting	Give your PC an address on the board's /24, such as <code>192.168.10.1/24</code> . Wait 5 minutes after a power-on, then <code>pyverify identify --host \$B</code> .
<code>info</code> says <code>rescue</code> after an install or a reset	No Linux on the user microSD, no microSD, or one of 2 GB or less	Put Linux on the card (§2.3, §2.4). Push or claim nothing else while it is in rescue.
<code>identify</code> times out, or the claim or <code>netboot</code> fails, while <code>info</code> works	Your PC's firewall drops the board's UDP replies	Allow UDP from the board on that Ethernet port.
<code>netboot</code> says "STAGE0's push tool is not at ..."	<code>pyverify</code> does not run from a platform checkout	<code>export MPS3_STAGE0_PUSH=<checkout>/src/linux_soc/hw/fw_stage0/stage0_push.py</code> (§2.4).
The claim is gone after a restart; SSH refuses your key	<code>/persist</code> was in memory: the claim was made before it was formatted, or while Linux ran from memory	Format <code>/persist</code> if needed, restart, claim again with <code>--replace-host-key</code> (§2.5).
Harness Manager refuses the board: the host key changed	A new card, a fresh <code>/persist</code> , or a restart with <code>/persist</code> in memory	Compare <code>board claim-status \$B</code> with <code>identify's host_key_sha256</code> , then <code>board claim \$B --replace-host-key</code> .
<code>Permission denied (publickey)</code> , or "claimed by another key" (exit 15)	The board was claimed with another key, or not from this Harness Manager	Use the key you claimed with; if it is yours, <code>board claim \$B --adopt</code> . Only <code>mps3-unclaim</code> at the harness serial console releases a claim.
<code>slot status</code> shows both slots <code>absent</code> , or a push is refused "slot B absent"	A blank or foreign card: no harness layout	Lay it out first (§2.4 step 3), or write it on your PC (§2.3).
"no permission to open serial port ..."	Your Linux user is not in the port's group	<code>sudo usermod -aG dialout \$USER</code> ; log out and in again.
No <code>V2M-MPS3</code> drive or no serial port	Another name on your system, or the Debug USB is not connected	<code>harness-manager probe</code> lists both (§2.2 has each system's names).
An SD install refuses: an earlier one was interrupted	The earlier write left a marker	Restore the backup, then install again. Never unplug or start a second write during one.
The board is <code>busy</code> , or exit 4	The control port takes one client: an open window, a console, a held <code>debug up</code> , or a script	Close the other client and retry. <code>harness-manager detach \$B</code> releases a lock your own terminal holds.

9.2 Loading a design

SYMPTOM	CAUSE	FIX
<code>program</code> or <code>restore</code> exits 15: <code>`mps3-debug down` failed before the swap (...)</code> ; the window offers Program anyway	Harness Manager could not stop OpenOCD on the board, so it programmed nothing	Retry. If it fails again, add <code>--force</code> , or tick Arm and click Program anyway (§3.2).
<code>program nanosoc_multicore</code> exits 15: "nothing was programmed: run it in a terminal and type the word..."	Its typed safety word was not given (§3.2)	Run it in a terminal and type <code>MULTICORE</code> , or pass <code>--allow-dut-flash-write</code> in a script.
<code>restore</code> : "unrecognized arguments: --yes"	<code>restore</code> asks nothing, so it has no <code>--yes</code>	Leave it out.
<code>restore</code> : "no baseline overlay (greybox) is known for the running shell"	No design directory holds <code>greybox</code>	Add it to <code>~/mps3-overlays</code> (§3.1).
Refused as INCOMPATIBLE (exit 14)	The design was built for another harness	Use this release's designs (reference F), or rebuild yours with this release's kit (chapter 7).
Refused with <code>identity lock: ...</code>	The harness's Linux and its FPGA disagree about which harness is loaded	Install this release's config SD and Linux image together (§2.7).
The Workbench does not list <code>nanosoc</code>	The release download carries only the open designs	Build it with the kit (§7.7). The nanoSoC source comes from SoC Labs, on proof of your Arm Academic Access (contact SoC Labs support).
The DUT does nothing after a harness restart	The partition stays in reset until the first load	Restore greybox, or load your design, once.
After a power-on the board loads a design you did not ask for	A design kept on the card (keep it on the card, or <code>pyverify deploy</code> without <code>-no-persist</code>)	<code>harness-manager card clear \$B</code> ; hold <code>USER_nPB1</code> at power-on to start on greybox once.

9.3 Consoles and debugging

SYMPTOM	CAUSE	FIX
The console is silent after a load	The program printed before you connected: output with no reader is lost	Open the console, then Ctrl-] r r to reset the DUT (§5.1).
Ctrl-C does not leave the console	Ctrl-C is sent to the board	Ctrl-] exits.
<code>nanosoc</code> prints at most a boot-ROM marker; GDB shows <code>pc 0xfffffffffe</code>	The DUT's flash holds a MicroPython boot image (known issue 14)	Use <code>nanosoc_upy</code> on that board (§4.6). <code>monitor reset halt</code> alone does not help (JTAG-DP STICKY ERROR); add <code>monitor nanosoc.cpu0 arp_examine</code> and <code>monitor halt</code> to stop it in the boot ROM.
GDB says <code>Servname not supported</code> , or the PC's own GDB starts	The attach line was pasted twice, or after <code>gdb</code>	Paste it once, alone, at an empty prompt (§5.2).
<code>nanosoc_upy</code> stopped starting after <code>nanosoc_multicore</code> was loaded	<code>nanosoc_multicore</code> wrote into the flash (known issue 15)	Write the MicroPython image again (§6.3).
Typed input is lost, or output has gaps	No receive buffer; bursts over 16 bytes	Let Harness Manager pace input; print short lines.
<code>reset</code> refused while a console is open	The console holds the control port	Reset from inside it: Ctrl-] r r.
<code>debug up</code> exits 13, "has no debug port"	Greybox or an open design is loaded	Load a nanoSoC design.

SYMPTOM	CAUSE	FIX
<code>debug up</code> exits 4, "the board's JTAG is held by ..."	Another debug session, from the window, another terminal or a PC OpenOCD	<code>debug status \$B</code> ; stop the other one; retry.
<code>debug up</code> exits 14, "no config for this design"	Only the published designs have a board configuration	Chapter 7, §7.6.
<code>debug up</code> or <code>xvc open</code> exits 15	This Harness Manager does not hold the claim, or the host key changed	<code>board claim \$B --adopt</code> , adding <code>--replace-host-key</code> after a new card.
<code>debug down</code> , <code>program</code> or <code>restore</code> refused: "your own `harness-manager debug up` (pid ...) holds it"	A terminal still holds a <code>debug up</code> session	Press Ctrl-C in that terminal, or <code>harness-manager detach \$B</code> .
The <code>debug up</code> terminal says <code>not reopened: the swap was not verified by the board</code>	The load did not verify	Ctrl-C; load again or restore greybox; then <code>debug up \$B</code> .
GDB fails to attach, with packet errors or a timeout	JTAG through SSH is slow; GDB's default timeout is 2 s	Use the <code>attach</code> line (<code>set remotetimeout 60</code>).
GDB: "Invalid register `xpsr`"	Arm GDB 13.3 does not use that name	<code>info registers</code> lists every register.
GDB takes minutes over a <code>load</code> or a memory dump	JTAG memory access is slow in v2.0.0	Build the program into the design (§6.3), or use v2.0.1 or later.
A PC OpenOCD or Vivado gets <code>jtag locked</code> or <code>xvc locked</code> , then a close	It connected to the claimed board directly	Use <code>debug up</code> or <code>xvc open</code> , or the SSH forwards of §5.3 and §8.4.
Vivado sees no XVC devices	An older <code>hw_server</code> ; one still lingering after a load; another XVC client	A 2026.1 <code>hw_server</code> , one client, and 20 s after each load.

Still stuck? Contact SoC Labs support (the support contact that came with your board), with the output of `harness-manager --json info $B` and `harness-manager version` , the service log `~/.config/harness-manager/daemon.log` , and what you ran.

10 Sharing boards with others

So far the board has been yours alone. This chapter covers the next steps: several boards on one network, and a group of people sharing boards through a **hub**. Everything in chapters 3 to 8 works the same on a shared board; this chapter covers only what changes.

10.1 Several boards on one network

Every new board answers at the same address, with the same MAC and the same name. Before two boards share a network, give each its own (§2.6):

```
harness-manager board identity 192.168.10.101 --label MPS3-A --mac 02:00:00:4D:50:61 --ip
192.168.10.111/24
# set up the second board alone on the network first, then:
harness-manager board identity 192.168.10.101 --label MPS3-B --mac 02:00:00:4D:50:62 --ip
192.168.10.112/24
```

- **One board at a time.** Connect a new board on its own, rename it, then add it to the shared network: two boards at the default address answer at once.
- **Harness Manager flags clashes:** `board identity` reports any other board with the same name, address or MAC.
- **DHCP.** The harness asks for a DHCP address first and keeps its static one as well, so it answers on both. `identify` reports the DHCP address.
- **Find the board in the room** with Identify (§2.6), and on the panel's top row.

A board of your own on a shared network is still yours: the claim guards its debugger, logic analysers and cards, but anyone on the network can load a design, reset the DUT or read its consoles. Agree who uses which board, or use a hub.

10.2 What a hub is

A hub is a Linux machine that holds each board's Debug USB and gives each board a private Ethernet link. It runs **fpgahub**, a service that keeps a queue of **leases** (who may use a board now), gives each board a stable name, and does the jobs that need the board's USB side: writing its config SD, rebooting its MCC, and sharing its serial consoles. Your tools reach each board through the hub, over SSH. Whoever runs the hub, the **hub administrator**, owns the boards' cards, claims and names; the *Technical Reference* describes running one.

From your hub administrator you need: an account on the hub (`<hub-host>`), in its group `fpga`, that logs in with your SSH key without a prompt; each board's target name and address; and your public key added to each board's claim, so that you can debug it.

10.3 Connect Harness Manager to the hub

```
harness-manager hub add myhub --ssh <hub-host>
harness-manager hub test myhub           # settings, reach, your key, the fpga group,
the boards
harness-manager hub targets myhub        # what the hub offers
harness-manager hub targets myhub --add mps3_01_pl  # add one board; prints its address
```

`hub test` stops at the first failure and names the fix; it never takes a lease. You still address the board by its own address; Harness Manager keeps one SSH connection to the hub that forwards the board's ports. In the window,

the same is under **Settings**: add a hub, test it, add a board, then open the board in the list. For one command without an entry, `--via ssh:<hub-host>` reaches a board, but takes no lease: use it only to read.

10.4 Leases and the house rules

A lease says whose turn it is. In the window each board behind a hub shows a lease badge, and opening it offers **Acquire lease**; while the board is open, Harness Manager renews the lease. From a terminal:

```
harness-manager lease acquire $B --ttl 3600      # waits in the queue until the board is yours
harness-manager lease show $B
harness-manager lease release $B
```

- **Asking for a board.** `lease request $B --message "..."` (or **Request board**) queues you and tells the holder, who can release it or keep it for 5 to 60 more minutes. With no answer in 2 minutes, and you first in the queue, `lease force $B` takes it: a last resort, which stops whatever the holder was running.
- **Leases are advisory.** Nothing on the board refuses a connection without one: they work because everyone takes one. Every session under one hub account counts as the same holder.

THE HOUSE RULES FOR A SHARED BOARD

1. **Take a lease first.** Reading the board with `info` needs none.
2. **Never claim a shared board.** The first key sent claims it for good and locks everyone else out. To debug, *adopt* the existing claim (§10.5); adopting adds no key.
3. **Never restart the harness, reboot the MCC, power-cycle, or write the cards or the DUT's flash.** Each changes the board for everyone. A DUT reset and a restore are fine.
4. **One client per port.** Close each tool when you finish, and close the board in Harness Manager.
5. **Leave the board on greybox and release the lease**, so the next person starts clean. Pass `--no-persist` to `pyverify deploy`.
6. **If the board is stuck, tell your hub administrator.** Your recovery stops at restoring greybox and a DUT reset.

10.5 Debugging on someone else's claim

A shared board's debugger and logic-analyser ports answer only through SSH with a key in its claim. Once your hub administrator has added your key, adopt the claim once; Harness Manager then jumps through the hub to the board for you:

```
harness-manager board claim $B --adopt --key ~/.ssh/id_ed25519.pub --yes
harness-manager debug up $B          # then GDB, as in §5.2
```

`debug up` and `xvc open` are for the lease holder (exit 4 without the lease). A board with no user microSD loses its claim and host key at every reset: until your key is added again, adopting fails with exit 15; afterwards, adopt again with `--replace-host-key`.

10.6 What else is different behind a hub

- **UDP does not cross the hub.** `identify`, the network boot and the window's **Scan** work only on the hub itself. Loads are not affected.
- **The Checks tab** appears for boards behind a hub: scripted board checks that hold the lease while they run.
- **By hand**, `pyverify` and other tools reach a shared board through an SSH tunnel to the hub, and SSH to the board itself through the hub with `-J <hub-host>`. Your hub administrator can give you the exact commands

for your hub.

- **Ask about each board:** whether it is for users, whether it has a user microSD (without one, it waits after any reset until the hub sends Linux again), whether your key is in its claim, and what is in its DUT flash.

SYMPTOM	CAUSE	FIX
Nothing answers through the hub	A link between you, the hub and the board	<code>harness-manager hub test myhub</code> names the failing step.
<code>Permission denied</code> on the hub's <code>fpgahub</code> socket	Your hub session started before you joined the <code>fpga</code> group	Log in to the hub again.
<code>info</code> says <code>rescue</code> , or <code>offline</code> for minutes	The board was reset and waits for the hub to send Linux	Tell your hub administrator; then restore greybox once and adopt the new host key.
Harness Manager lost its lease mid-session	Someone took it after a request, or another session on your account did	<code>lease show \$B</code> names the holder; wait until the board is yours.

A Harness Manager commands

The look-up for Harness Manager 1.0.0. Every command is `harness-manager <verb> ...`; `harness-manager <verb> --help` prints every option, and `harness-manager help --tabs` the window's help.

Global: `--json` or `--tsv` (before the verb), `--version`. **Board:** `TARGET` is the board's address, `host[:port]`, or `-` for a board reached only over the Debug USB; `--serial PORT` and `--volume PATH` add the MCC's serial port and the mounted config SD; `--via ssh:HOST|hub` reaches a board through an SSH tunnel or its hub. Rows marked **own board** change a board's cards, controller or harness: never on a shared board.

A.1 Commands

COMMAND	PURPOSE
The board	
<code>version · probe · info TARGET</code>	Harness Manager's version; look for boards (USB ports and the default address); what a board is, its health (A.4) and what works.
<code>telemetry TARGET · clock TARGET</code>	Every reading, with its source; the clocks (the DUT clock is fixed at 50 MHz on v2.0).
<code>identify TARGET [--seconds N]</code>	Blink the panel and show IDENTIFY for 1–30 s (default 5; 0 stops). At most once every 10 s.
<code>attach · detach TARGET</code>	Hold Harness Manager's session lock on the board in the foreground; release a lock your own <code>attach</code> , <code>console</code> or <code>debug up</code> holds.
Designs	
<code>overlays TARGET [--overlay-dir DIR]</code>	The designs that load on this board, and why any other does not.
<code>program TARGET RM [--yes] [--keep-on-card] [--force] [--allow-dut-flash-write] [--overlay-dir DIR]</code>	Load a design: preflight, confirm, push, verify. RM is a name or <code>rm_id</code> . <code>--force</code> : swap even if the board's OpenOCD could not be stopped (§3.2). <code>--allow-dut-flash-write</code> : the typed word for <code>nanosoc_multicore</code> , for scripts.
<code>restore TARGET [--force] [--overlay-dir DIR]</code>	Load greybox ("the baseline") and confirm it. No question, no <code>--yes</code> .
<code>reset TARGET [WHAT]</code>	Reset the DUT (the default).
Consoles, debugging, the panel	
<code>console TARGET NAME [--read-only] [--export PORT] [--for S]</code>	A console in this terminal: <code>uart0</code> , <code>uart1</code> , <code>swo</code> . Ctrl-] exits; Ctrl-] then r, r resets the DUT.
<code>pty TARGET NAME · baud TARGET NAME [RATE]</code>	A PTY for <code>screen</code> on the same console; a console's baud rate.
<code>debug up down status detect TARGET</code>	<code>up</code> starts OpenOCD on the board and prints one GDB port and <code>attach</code> line per core; holds until Ctrl-C or <code>--for S</code> . <code>detect</code> reads the IDCODE without halting.
<code>xvc open close status tcl ltx TARGET</code>	Logic analysers over XVC: open (<code>--byo</code> : your own <code>hw_server</code>), the Vivado Tcl, the probe file (<code>-o FILE</code>), close.
<code>display TARGET snapshot status show · panel show mirror TARGET</code>	The panel, pixel for pixel or as text.
<code>lab TARGET link display macgen dutr</code>	Test tools for a DUT's Ethernet: link events, traffic generator, frames the DUT sent.
The Linux harness: claim, identity, user microSD	
<code>board claim TARGET [--key PUB] [--adopt] [--replace-host-key] [--yes]</code>	Claim with your key (own board); <code>--adopt</code> pins a claim that already trusts your key; <code>--replace-host-key</code> accepts a new host key.

COMMAND	PURPOSE
<code>board claim-status TARGET · board ssh TARGET [-c CMD] [--print]</code>	Is it claimed, and by whom; a root shell or one command on the board.
<code>board identity TARGET [--label L] [--ip A/N] [--mac M] [--hostname H] [--unset F] [--clear]</code>	The board's name, address and MAC, and any clash; with options, change them (own board ; a typed phrase, or <code>--consent</code> ; a 3-minute restart).
<code>card status commit clear TARGET</code>	The design kept for power-on (own board , except status).
<code>slot status push commit rollback verify TARGET</code>	The OS slots: <code>push --bundle DIR</code> (or <code>IMAGE --static-id ID</code>), about 45 min (own board , except status).
Controller, config SD, power (own board)	
<code>mcc TARGET temp osc reboot cmd LINE</code>	The MCC: temperatures, oscillators, a reboot it proves down and up (up to 300 s), one allow-listed command.
<code>sd TARGET backup DIR install BUNDLE --backup ZIP restore ZIP</code>	The config SD over the Debug USB. <code>install</code> refuses without a backup, never writes MCC firmware, and refuses a bundle that would make the MCC update itself (<code>--allow-mcc-update</code>).
<code>power show cycle TARGET</code>	A networked power plug named in <code>boards.toml</code> .
Building	
<code>kit import info list fetch verify</code>	The build kit for a harness (<code>--static-id ID</code>).
<code>kit guide script build check pack</code>	Where you are; write the build directory (<code>--design F --out DIR</code>); print the Vivado command (<code>--gui</code>); check the receipt and the boundary timing; make the design (<code>--import</code>).
<code>xdc info rm-kit board</code>	Constraint files: the built-in designs' groups; the partition kit and wrapper skeleton; a whole-board export.
Sharing (chapter 10)	
<code>hub add test targets list adopt token remove</code>	Define a hub, test it (no lease), add its boards (<code>targets NAME --add T</code>).
<code>lease show acquire release request requests respond force leave dismiss TARGET</code>	The hub lease and its queue.
<code>share list start TARGET</code>	The hub's serial shares of the board's UART lanes.
The window, the service, settings, updates	
<code>ui [--no-browser] [--demo] · app [--demo]</code>	Open the window in a browser or its own window; <code>--demo</code> shows virtual boards.
<code>daemon start stop status</code>	The background service (one per user).
<code>config list get set unset path test · config set-secret KEY</code>	Settings (A.2).
<code>harness ... · update ...</code>	The signed release channel: not live in 1.0.0. Update by hand (§2.7).

A.2 Files and settings

The installer puts one command, `harness-manager` , in `~/.local/bin` (Windows: `%LOCALAPPDATA%\harness-manager\bin`) and a private environment in `~/.local/share/harness-manager/venv` . Your state is in `~/.config/harness-manager: boards.toml` (how to reach each board, its pinned host key, a power plug, its hub), `settings.toml` , `daemon.log` (the service's log), config-SD backups and build kits. Put `~/.local/bin` on the PATH in `~/.bash_profile` or `~/.profile` , not only `~/.bashrc` . Settings you are most likely to change:

KEY (DEFAULT)	WHAT IT SETS
<code>mps3.overlay_dirs</code>	Your design directories
<code>debug.on_board</code> (<code>auto</code>)	Where OpenOCD runs: on the board when it can, else this PC; <code>true</code> , on the board or refuse; <code>false</code> , this PC only
<code>tools.vivado</code> , <code>tools.hw_server</code> , <code>tools.openocd</code>	The tools, found on <code>PATH</code> when empty; <code>tools.openocd</code> is only for the PC fallback
<code>mps3.console.pace_ms</code> (20)	Delay between characters typed to the DUT
<code>panel.identify_s</code> (5)	How long Identify blinks
<code>debug.port_base</code> , <code>debug.xvc_port_base</code> (0)	Pin the local GDB and XVC ports (0: a free one)

An environment variable `HARNESS_MANAGER_<KEY>` overrides a setting for the life of the service (`daemon status` lists them). After changing a tool path, run `daemon stop`.

A.3 Exit codes

CODE	NAME	MEANING
0	OK	Done.
1	FAILED	An internal failure: a bug. The message says so.
2	USAGE	Bad arguments, or a target spelt wrongly.
3	ABSENT	No such board, design or port.
4	HELD	Someone else holds it: the lease, the session lock or a one-client port.
5	PORT_BOUND	A local port Harness Manager needs is in use.
6	ACTION_FAILED	The board refused, or the action did not complete.
7	UNREACHABLE	No route, a timeout, or the connection was refused.
8	ALREADY	Already in the requested state.
12	UNAVAILABLE	The capability is missing on this board or over these links; the message says what it needs.
13	NOTHING_ON_TARGET	The link works, but nothing answered (for example, no debug port in the design).
14	INCOMPATIBLE	An identity does not match: <code>static_id</code> , <code>USERCODE</code> or protocol.
15	REFUSED	A safety rule: a missing backup or confirmation, a destructive MCC command, another key's claim, a load whose first step could not stop the board's OpenOCD, or <code>nanosoc_multicore</code> without its typed word.

A.4 Health states

idle	The harness answers; ready.
busy	Another client holds the control port, or a load is running.
wedged	The harness took the connection and never replied: recover (§2.8).
offline	Nothing answers. With a note: the board answers identify or SSH, but <code>harnessd</code> is not listening; it may be restarting.
rescue	No Linux running: the boot loader waits for an image (§2.4).

B pyverify

`pyverify` (package `mps3-pyverify`, in `host/pyverify/` of the platform repository) is the Python library and command line beneath Harness Manager. Run it as `python3 -m pyverify.cli <verb>`; this guide writes `pyverify <verb>` (§8.2). Every board verb takes `--host ADDR`; `pyverify <verb> --help` gives the rest.

COMMAND	PURPOSE
<code>identify --host H</code>	The UDP probe: mode (<code>run</code> , <code>rescue</code>), claim, host-key fingerprint, label. Answers even in rescue; local network only.
<code>ping · version · stats · diag · log</code>	Read the board: identities; the harness release and features; the board in one line; counters; the harness's log.
<code>deploy --host H --overlay DIR [--no-persist]</code>	Load a design and verify it. Keeps it on the card unless <code>--no-persist</code> . Exit 0 verified, 1 refused, 2 bad manifest, 3 network failure.
<code>reset --host H</code>	Reset the DUT.
<code>dut-console --host H [--send LINE] [--seconds S]</code>	Read the DUT console, or type a line (paced at 20 ms per character) and read the answer.
<code>dut-eth --host H [--frames N] · display --host H</code>	Frames the DUT sent; who drives the panel.
<code>claim --host H --key K.pub</code>	Claim an unclaimed board (own board). Exit 0 accepted, 1 already claimed, 2 not a public key, 3 unreachable.
<code>netboot IMAGE --host H [--tool PATH] · netboot --status</code>	Send Linux to a board in rescue; it boots once, from memory. Needs the push tool (<code>--tool</code> or <code>MPS3_STAGE0_PUSH</code> , §2.4). Exit 0 accepted, 1 bad image, 2 refused, 3 not in rescue, 4 transfer failed.
<code>slot status push commit rollback verify</code>	The OS slots (own board).
<code>usd status format clear rescan --host H</code>	The card's design store (own board , except <code>status</code>).
<code>reboot --host H [--wait]</code>	Restart the harness's Linux (own board).
<code>lease acquire release status preflight heartbeat cancel</code>	The hub lease (chapter 10). <code>acquire</code> prints a bare token.

The Python API. `Mps3Board(host, *, overlay_root=...)` is a context manager; `deploy(name_or_dir, persist=True)` returns a result with `.rm_id`, `.verified` and `.reattach`; `reset("dut")`; `uart0`, `uart1` and `swo` are console readers with `read_until(pattern, timeout=)`, `assert_contains(pattern, timeout=)` and `write_line(text)` (paced). A refusal by the harness comes back in the reply, not as an exception. `pyverify.testing.fakeshell.FakeShell` models the harness's network side for tests with no board.

C Ports and protocol

Everything a host does to the board goes through its one 10/100 Ethernet port, using the harness's network protocol (v0.17). `<ip>` is the board's address, `192.168.10.101` by default. The protocol contract, `docs/contracts/net-protocol.md` in the platform repository, has every field.

PORT	SERVICE	CLIENTS	ON A CLAIMED BOARD
6900/tcp	Control: one JSON object per line, request then reply	One	Open, except changes to the cards and the board's identity, which go through SSH

PORT	SERVICE	CLIENTS	ON A CLAIMED BOARD
6910/tcp	Design upload, during a load	One	Open for designs; OS images only through SSH
69/udp	TFTP: uploads, the claim, the rescue boot	Local network	Designs open; images and claims refused
6899/udp	Identify; answered in rescue too	Local network	Open
22/tcp	SSH, key-only	Many	The claim's keys only
6921/tcp	The board's JTAG port, used by OpenOCD on the board (or on your PC)	One	From the board itself only: forward it over SSH
3333 , 3334 , 4444 , 6666	OpenOCD on the board: GDB per core, telnet, Tcl	One debugger	The board's own loopback only; Harness Manager forwards GDB
2542/tcp	XVC to the partition's debug bridge	One	From the board itself only
6930/tcp	DUT console (UART0)	One	Open
6931/tcp	DUT UART1: inert in the published designs	One	Open
6932/tcp	SWO trace	One	Open
6940/tcp	Panel mirror, for Harness Manager's Front panel	Two	From the board only; Harness Manager forwards it

- **One client per port** for 6900, 6910, 6921 and 2542: a second connection is reset. A tool that connects straight after another closed may be refused; retry, and leave 2 s between OpenOCD sessions.
- **UDP stays on the local network:** identify and TFTP do not cross an SSH tunnel or a router.
- **The console drops what nobody reads**, and every load flushes it. Type at most one character per 20 ms.
- **Addresses:** the harness asks for DHCP and always keeps its static address too (`192.168.10.101/24` unless renamed, §2.6).
- **By hand:** `echo '{"op":"ping"}' | nc <ip> 6900` answers `{"ok":true,"shell_id":"0x44ee76d5","rm_id":"0x00000000"}`.

D Identities and versions

A board, its harness and a design each carry several identities, because each answers a different question.

IDENTITY	ANSWERS	V2.0	READ IT WITH
static_id	Will this design fit this harness? A checksum of the harness's locked FPGA layout	<code>0x44EE76D5</code>	<code>info</code> , the panel's <code>shell</code> row, each design's <code>manifest.json</code>
rm_id	Which design is loaded? Version major, minor, design number; 0 is greybox	Per design: reference F	<code>info</code> , <code>pyverify stats (rm)</code>
USERCODE	Which build run of the harness?	<code>0xFB1F8C76</code>	JTAG only; <code>static_usercode</code> in each manifest
Harness software	Which release of the harness's software and FPGA image?	<code>1.0.0</code>	<code>info</code> , <code>pyverify version</code>
Linux	The harness's operating system	Linux 6.18 on MicroBlaze V	<code>pyverify version (sha)</code>
Protocol	Which requests does the harness answer?	v0.17	<code>pyverify version (features, by name)</code>

IDENTITY	ANSWERS	V2.0	READ IT WITH
Board identity	Which board is this? Name, address, MAC	New board: <code>MPS3</code> , <code>192.168.10.101</code> , <code>02:00:00:4D:50:53</code>	The panel's top row, <code>board identity</code> , <code>identify</code>
Platform release	Which release of the whole platform?	v2.0.0	The release at https://github.com/SoC-Labs/MPS3-NanoSoC-Verification-Platform/releases
Tools	Which host tools?	Harness Manager 1.0.1	<code>harness-manager version</code>

- **Why the harness says 1.0.0 in platform v2.0.0:** the harness's version is stamped into its FPGA image and changes only when the harness itself is rebuilt with a new `static_id` . Use the `static_id` to tell harnesses apart.
- **When identities disagree** (a Linux image built for another harness), every load is refused with `identity lock: ...` . Install the config SD and the Linux image of the same release.
- **Board identity** comes, field by field, from the board's own setting on the user microSD (`board identity`), else the defaults above. A board without a user microSD keeps the defaults.

E Boundary signals

The wires between the harness and your design: the ports your wrapper must declare, exactly (chapter 7). The source is `fpga/shell/boundary.yaml` in the platform repository. **In** means an input to your design; **out**, an output of it.

GROUP	SIGNALS	BITS
clkrst	<code>dut_clk</code> in (50 MHz); <code>dut_resetrn</code> in, the DUT reset; <code>rp_resetrn</code> in, held through a load; <code>dbg_resetrn</code> in, the debugger's reset	4
jtag	<code>jtag_tck</code> , <code>jtag_tms</code> , <code>jtag_tdi</code> in; <code>jtag_tdo</code> out	4
dbgbscan	Eleven <code>dbg_bscan_*</code> legs in from the harness's debug bridge (<code>capture</code> , <code>drck</code> , <code>reset</code> , <code>runtest</code> , <code>sel</code> , <code>shift</code> , <code>tck</code> , <code>tdi</code> , <code>tms</code> , <code>update</code> , <code>bscanid_en</code>); <code>dbg_bscan_tdo</code> out, 0 without ILAs	12
eth	RMII: <code>phy_rmii_ref_clk</code> in (50 MHz), <code>phy_rmii_crs_dv</code> , <code>phy_rmii_rxd[1:0]</code> in; <code>phy_rmii_txd[1:0]</code> , <code>phy_rmii_tx_en</code> out. MDIO: <code>mdc</code> , <code>mdio_o</code> , <code>mdio_oe</code> out; <code>mdio_i</code> in	11
uart	Console out: <code>uart_tx_tdata[7:0]</code> , <code>uart_tx_tvalid</code> out, <code>uart_tx_tready</code> in. Console in: <code>uart_rx_tdata[7:0]</code> , <code>uart_rx_tvalid</code> in, <code>uart_rx_tready</code> out. <code>sw0</code> out	21
status	<code>rm_id[31:0]</code> out; <code>dut_lockup</code> out; <code>irq_out</code> out	34
gpio	<code>dut_gpio_o[15:0]</code> , <code>dut_gpio_oe[15:0]</code> out; <code>dut_gpio_i[15:0]</code> in: LEDs on 7:0, switches on 15:8	48
qspi	<code>qspi_sclk</code> , <code>qspi_csn</code> , <code>qspi_io_o[3:0]</code> , <code>qspi_io_oe[3:0]</code> out; <code>qspi_io_i[3:0]</code> in: the board's flash, passed straight to the pins	14
Total: 47 ports		148

During a load the harness clamps every output of your design to 0, except `qspi_csn` , held at 1. The QSPI group is the one group the harness does not synchronise.

F The design catalogue

The 13 designs of platform v2.0, all built for `static_id 0x44EE76D5`. Eight are open and come built in the release download. Five contain Arm IP: the download does not carry them, and you build them yourself from the nanoSoC source with the kit (§7.7).

DESIGN	RM_ID	WHAT IT IS	LOAD
Open: no Arm IP			
greybox	0x00000000	The empty design: every output tied off. What <code>restore</code> loads	~40 s
led	0x0100001E	The LEDs count up: a visible check that a load changed the board	~35 s
uart_echo	0x01000004	Prints <code>rm_uart_echo ready</code> on the console and echoes what you type	~40 s
regdemo_a, regdemo_b	0x010000A1 , 0x010000B2	A pair that differ in one register value: proof that a load really changed the design	~40 s
clcd_demo	0x00010007	Draws a test card on the board's display, from the DUT	~40 s
dbg_demo	0x01000009	A counter with a logic analyser: the reference design for XVC; with an <code>.ltx</code>	~40 s
socscope	0x01000006	A self-test of the SoCScope trace path, on the SWO port	~65 s
Arm IP: built by you, with Arm Academic Access (§7.7)			
nanosoc	0x01000001	One Cortex-M0 with a hello program built in (chapter 4). Not on a flash that holds MicroPython	~65 s
nanosoc_ila	0x0100000A	<code>nanosoc</code> with logic analysers on its console, GPIO, JTAG and reset; with an <code>.ltx</code>	~75 s
nanosoc_upy	0x01000005	The nanoSoC with 128 KB of IMEM, booting MicroPython from the DUT's flash	~75 s
nanosoc_multicore	0x01000003	Two Cortex-M0+ cores, Ethernet MAC and PTP clock, DMA, messaging, QSPI. Writes the DUT's flash (§3.2); only core 1 can be debugged	~85 s
eth_ss	0x01000002	An Ethernet MAC and PTP subsystem with no CPU, on CMSDK bus blocks; sends a PAUSE frame every second	~50 s

Each design is a directory: `manifest.json`, `<design>.bin` (the partial), `<design>_clear.bin` (the clearing) and, for the logic-analyser designs, `<design>.ltx`. Designs from earlier platform releases do not load on v2.0. A design you build yourself gets its own `rm_id` (chapter 7).

G Known limitations in v2.0.0

What does not work yet, or works with a limit, in platform v2.0.0, and what to do about each. Each also appears where it matters in the chapters.

LIMITATION	WHAT IT MEANS	WHAT TO DO
Slow JTAG memory access	GDB reads and writes memory at about 20 bytes per second; a halt with a register read takes about 50 s.	Build programs into the design (§6.3). v2.0.1 or later is much faster.
Boundary timing (known issue 11)	Builds from an older build script, or from a synthesis checkpoint you bring, can leave the paths between the harness and your design un-analysed. The published v2.0 designs have been re-timed and meet timing on the boundary.	Build with Harness Manager 1.0.0's <code>kit script</code> ; <code>kit check</code> says whether your build was timed (§7.5).

LIMITATION	WHAT IT MEANS	WHAT TO DO
nanosoc and a MicroPython flash (known issue 14)	Plain nanosoc locks up if the DUT's flash holds a MicroPython boot image.	Use nanosoc_upy on that board. A flash-erase step comes later.
nanosoc_multicore writes the flash (known issue 15)	It writes one byte at 0x20000 on every boot, which damages a MicroPython image. Harness Manager asks you to type MULTICORE.	Do not load it on a board whose flash holds MicroPython.
nanosoc_multicore debug	Only core 1 can be debugged; the board refuses core 0.	Debug the single-core designs.
Console bursts	On the nanoSoC designs the console drops bytes in bursts longer than 16.	Print short lines.
Kept design not loaded at power-on	The card is read too slowly at power-on (about 130 s for nanosoc , against a 30 s limit): card status shows power-on failed:timeout (§3.5).	After a power cycle, load the design again; the kept pair stays on the card.
Programs loaded with GDB are silent	A program loaded into IMEM with GDB runs, but its console output does not yet reach the DUT console (<i>Getting Started</i> ch 13).	Check the program with GDB: read its variables and registers.
Console before reset	Output with no reader is lost, so a banner printed at reset reaches only a console already open.	Open the console, then reset the DUT (Ctrl-] r r).
nanoSoC GPIO outputs	The nanoSoC designs' GPIO 0 outputs stay at 0, so their LEDs do not follow the program.	Use the console; the open led design shows the LEDs work.
Panel names	The front panel shows some designs as rm?<id> , such as rm?01000005 for nanosoc_upy .	None: info shows the name.
DUT clock	Fixed at 50 MHz.	Build for 50 MHz.
Debug configurations	The board's OpenOCD speaks JTAG only, and has configurations only for the published designs.	Give your design a JTAG port; see §8.4 for your own designs.
Board revision B	Revision B boards boot, but are untested; revision C is supported.	Report problems on revision B to SoC Labs support.
Not yet run on a real board	Writing the user microSD on a PC (§2.3); keeping a design on the card (§3.5); logic analysers over XVC (§5.4); the board's OpenOCD by hand (§5.3); loading a program with GDB (§6.3).	Keep your config-SD backup; use the proven routes first; tell SoC Labs support what you see.
Updates	Harness Manager's built-in update commands are not live.	Update by hand (§2.7).

H The nanoSoC memory map

Where things sit in the single-core nanoSoC of the **nanosoc** design. For a SoC you generate yourself, the generated **nanosoc_memmap.h** is the authority: check it before you write addresses into firmware.

REGION	BASE	NOTES
Boot ROM	0x0000_0000	Runs at reset; REMAP puts IMEM here after boot (the M0 has no vector-table offset register)
IMEM	0x1000_0000	Instruction memory: 16 KB in nanosoc , 128 KB in nanosoc_upy
DMEM	0x1800_0000	Data memory: 16 KB in nanosoc
APB peripherals	0x4000_0000	Timers, dual timer, build information, UARTs (console UART2 at 0x4000_6000), watchdog

REGION	BASE	NOTES
GPIO 0, GPIO 1	0x4001_0000 , 0x4001_1000	GPIO 0: LEDs on bits 7:0, switches on 15:8. GPIO 1: internal (the console's pins)
System control	0x4001_F000	REMAP ; BOOT_CFG at +0x14
Expansion	0x6000_0000	256 MB AHB port: the slot for your accelerator
QSPI memory, control	0x7000_0000 , 0x7400_0000	Execute-in-place window onto the flash (4 MB); the controller and its cache
SRAM 0, SRAM 1	0x8000_0000 , 0x9000_0000	Spare RAM banks
CPU debug window	0xA000_0000	Reachable from the debug port only

Glossary

Adopt (a claim) `board claim --adopt`: log in with a key the board already trusts and pin its host key. It adds no key, so it is not claiming (§2.5, §10.5).

Arm-IP design, open design A design that contains Arm IP (the nanoSoC designs, `eth_ss`), which holders of Arm Academic Access build themselves (§7.7); and one that contains none, which comes built in the release download (reference F).

Attach line The GDB command `debug up` prints: `arm-none-eabi-gdb` with `set remotetimeout 60` and one core's local port. Paste it (§5.2).

Baseline Harness Manager's word for greybox.

Board identity A board's name (label), address, MAC and host name (§2.6). Not the harness's identities.

Boot ROM nanoSoC's first program, at address 0: it hands over to IMEM, or loads a program from the flash (§4.4).

Boundary The fixed set of wires between the harness and the partition (reference E).

Build kit What you build a design against for one harness: its locked checkpoint, the boundary and settings (chapter 7).

Claim The one-time act that decides which SSH key a board trusts: the first key sent to it. It lasts until released at the harness serial console (§2.5).

Clearing The file of a design that blanks the partition before the next design.

CMSDK Arm's Cortex-M System Design Kit: the bus and peripheral blocks nanoSoC is built from.

Config SD The MCC's card, which holds the harness; the `V2M-MPS3` drive over the Debug USB.

Cortex-M0 Arm's smallest Cortex-M processor: the CPU of this release's single-core nanoSoC designs. `nanosoc_multicore` has two Cortex-M0+.

Debug USB The board's USB port to its MCC: four serial ports and the config SD. For installing and for the MCC only.

Design (overlay) What you load into the partition: a partial, a clearing and a manifest (§1.2).

DFX Dynamic Function eXchange: AMD's partial reconfiguration, which rewrites the partition while the harness runs.

DUT Device under test: the design loaded in the partition.

fpgahub The service a hub runs to share boards: leases, names, and the boards' USB side (chapter 10).

greybox The empty design, `rm_id 0`: the safe state to leave a board in.

Harness The permanent FPGA design and its Linux: network, consoles, debugger, panel.

harnessd The harness's service that answers the network.

Harness Manager The window, command line and background service you drive boards from (reference A).

Hub, hub administrator A machine that runs fpgahub and holds shared boards; the person who runs it (chapter 10).

identify The UDP probe on port 6899: what answers at an address, its mode, claim and host key. Local network only. Not the Identify button, which blinks the panel.

ILA, .ltx An AMD logic analyser inside a design, and the probe file that describes it (§5.4).

IMEM, DMEM nanoSoC's instruction and data memories.

Keep on the card Writing a loaded design to the user microSD, so the board loads it at power-on (§3.5).

Lease A queued, time-limited turn on a shared board, from fpgahub. Advisory; not a claim.

MCC The board's Motherboard Configuration Controller: it powers the board and loads the FPGA from the config SD.

MicroBlaze V AMD's RISC-V soft processor: the harness's CPU, running Linux.

mps3-debug The harness's launcher for its OpenOCD, run over SSH (§5.3).

nanoSoC SoC Labs' family of Arm Cortex-M SoCs built on an AHB bus fabric (chapter 4).

OS slot One of the two Linux copies, A and B, on the user microSD (§2.7).

Partial The file of a design that holds the design itself.

Partition The region of the FPGA that holds one design at a time.

/persist The user microSD partition that keeps the claim, the board's name and its SSH host key.

Preflight The checks before a load. A failure writes nothing.

pyverify The Python library and command line beneath Harness Manager (reference B).

QSPI Quad SPI: the interface to the board's flash, which the DUT owns.

Rescue The boot loader's waiting mode when there is no Linux to boot (§2.4).

rm_id The 32-bit identity a design drives; read back after a load (§1.2).

static_id The harness's fingerprint: a design loads only on the harness whose **static_id** it names.

Swap A load: decouple, clear, write the partial, verify (§3.3).

SWJ-DP The CoreSight debug port in nanoSoC; the harness uses it in JTAG mode.

User microSD The harness's own card: OS slots, **/persist** and the design store.

Verified A load whose **rm_id** the harness read back as expected.

XiP Execute in place: running code straight from the QSPI flash.

XVC Xilinx Virtual Cable: JTAG over TCP, so Vivado reaches the logic analysers (port 2542).